Introduction to software engineering

Deliverable 5

Adham Sobhy - 23-101003

Yassin Farrag 23-101019

Andrew Emad 23-101092

Ahmed 25-102360

Contents

1. Introduction 3
2. Assumptions & Design Rules 4
   2.1 Layered Architecture Rules 4
   2.2 Array-Based Persistence Requirement 4
   2.3 External Services as Interfaces 4
   2.4 Status Representation 4
   2.5 Error Handling Style 4
   2.6 Scope 4
3. Use Case Classification (Simple / Moderate / Complex) 5
   3.1 Classification Criteria 5
   3.2 Classification Summary 5
4. Selected 12 Core Use Cases (4 per category) 5
5. OO Design Method (Chapters 12 & 13) 6
6. Use Case Realizations (12 Use Cases) 6
   6.1 UC-02 — Register Patient Information (Simple — CRC) 7
   6.2 UC-06 — Cancel Appointment (Simple — CRC) 8
   6.3 UC-09 — Send Appointment Reminders (Simple — CRC) 11
   6.4 UC-T2 — Update Slot Availability (Simple — CRC) 13
   6.5 UC-04 — Confirm Appointment Slot (Moderate — Communication) 14
   6.6 UC-08 — Authorize Patient Discharge (Moderate — Communication) 16

6.7 UC-11 — Generate Monthly Bill (Moderate — Communication) 19
6.8 UC-T3 — Synchronize Patient Information (Moderate — Communication) 21
6.9 UC-01 — Book Appointment (Complex — Sequence) 23
6.10 UC-03 — Manage Doctor Schedule (Complex — Sequence) 26
6.11 UC-05 — Complete Patient Treatments (Complex — Sequence) 30
6.12 UC-07 — Make Payment (Complex — Sequence) 32
7. Final Consolidated Design Class Diagram (DCD) 34
8. Package Diagram (Architecture) 36
9. Implementation (StarUML → Java + Walking Skeleton, Arrays Only) 37
9.1 Model-to-Code Transformation 37
9.2 Implemented Use Cases 39
9.3 “Database Interaction” via Array-Based DAOs 40
10. Evidence Checklist (Screenshots) 42
11. Jira / GitHub Evidence 43
12. Conclusion 43

1. Introduction

This deliverable presents the detailed object-oriented design of the Healthcare Management System (HMS) using the use-case driven approach taught in Chapters 12 and 13. The design work proceeds use case by use case, transforming analysis artifacts prepared in earlier deliverables into design-level classes, responsibilities, and method signatures.

We classified the system's use cases into simple, moderate, and complex categories. Based on the course requirement, we selected 12 core use cases (4 per category) and performed use case realizations. Each realization begins from a first-cut layered partition and then applies the required technique according to the use-case type:

- Simple: CRC (Class–Responsibility–Collaboration)
- Moderate: Multi-layer communication diagrams
- Complex: Multi-layer sequence diagrams

Finally, we performed StarUML model-to-code transformation (Java generation) and implemented a runnable walking skeleton in NetBeans for:

- UC-02 Register Patient Information
- UC-06 Cancel Appointment

Implementation follows strict three-layer architecture: View → Controller → DAO and uses array-based DAO persistence (no database), as instructed.

---

2. Assumptions & Design Rules

2.1 Layered Architecture Rules

- View Layer (ui) contains boundary/UI classes only.
- Controller Layer (controller) mediates all communication between View and other layers.
- Data Access Layer (dao) encapsulates persistence operations and simulated “database interaction”.
- Domain Layer (domain) contains entity classes representing business state.
- Direct View → DAO calls are forbidden.

2.2 Array-Based Persistence Requirement

“Database interaction” is satisfied using arrays stored inside DAO classes. Only DAO classes read/write these arrays.

2.3 External Services as Interfaces

External systems are represented as interfaces (NotificationService, PaymentGateway, ExternalPatientSystem) and invoked only from controller classes.

2.4 Status Representation

Statuses are modeled as strings (e.g., CANCELLED, CONFIRMED, PAID) for UML + implementation consistency.

2.5 Error Handling Style

Controllers return clear success/error outputs suitable for a walking skeleton. Alternate flows are handled through controller validation.

2.6 Scope

All 12 use cases are designed and realized. Implementation focuses only on UC-02 and UC-06.

---

3. Use Case Classification (Simple / Moderate / Complex)

3.1 Classification Criteria

- Simple: CRUD / single goal; limited collaborators; minimal alternate flows.
- Moderate: Multi-object collaboration across layers; validations; state changes.
- Complex: Many interacting objects/services; multiple alternate flows; orchestration and external integration.

3.2 Classification Summary

- Simple (CRC): UC-02, UC-06, UC-09, UC-T2
- Moderate (Communication): UC-04, UC-08, UC-11, UC-T3
- Complex (Sequence): UC-01, UC-03, UC-05, UC-07

---

4. Selected 12 Core Use Cases (4 per category)

UC ID	Use Case Name	Primary Actor	Category	Justification
UC-01	Book Appointment	Patient / Receptionist	Complex	Orchestration + slot validation + alternate paths
UC-02	Register Patient Information	Receptionist	Simple	CRUD + validation + persistence
UC-03	Manage Doctor Schedule	Doctor	Complex	Conflict detection + schedule updates
UC-04	Confirm Appointment Slot	Receptionist	Moderate	Status transition + validation + messaging
UC-05	Complete Patient Treatments	Doctor	Complex	Record update + optional prescription + billing trigger
UC-06	Cancel Appointment	Patient / Receptionist	Simple	Status update + rule checks + notification

UC ID	Use Case Name	Primary Actor	Category	Justification
UC-07	Make Payment	Patient	Complex	External payment gateway + failure handling
UC-08	Authorize Patient Discharge	Doctor	Moderate	Rule approval + record update + room release
UC-09	Send Appointment Reminders	Scheduler	Simple	Scheduled notifications
UC-11	Generate Monthly Bill	Scheduler	Moderate	Aggregation + bill creation + alternate flow
UC-T2	Update Slot Availability	Scheduler	Simple	Scheduled scan + status update
UC-T3	Synchronize Patient Information	Receptionist/System	Moderate	External sync + failure handling

---

5. OO Design Method (Chapters 12 & 13)

For each selected use case, we followed a consistent OO design loop:

1. Build a first-cut partition (Boundary / Controller / Entity / DAO / External).
2. Apply the required realization technique (CRC / Communication / Sequence).
3. Extract messages from the realization and assign them to class operations.
4. Update the DCD and repeat until the final DCD is complete and consistent with layering rules.

---

6. Use Case Realizations (12 Use Cases)

For each use case, we include:

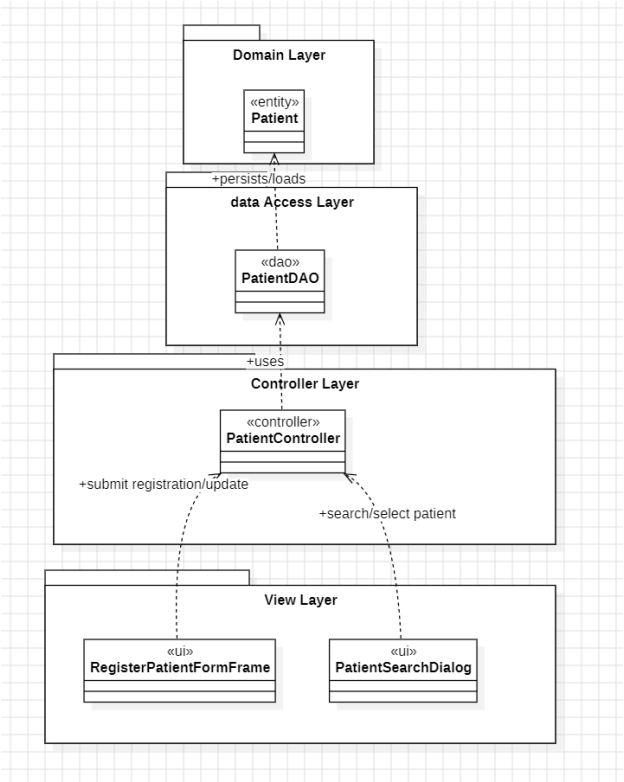
- First-cut partition (Before)
- Realization diagram

- Updated partition (After)
- Extracted operations table

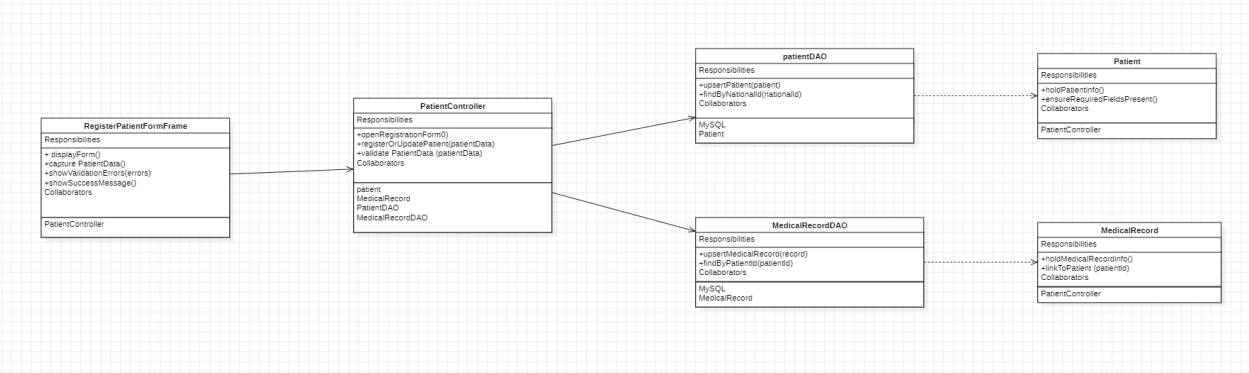
6.1 UC-02 — Register Patient Information (Simple — CRC)

Figures

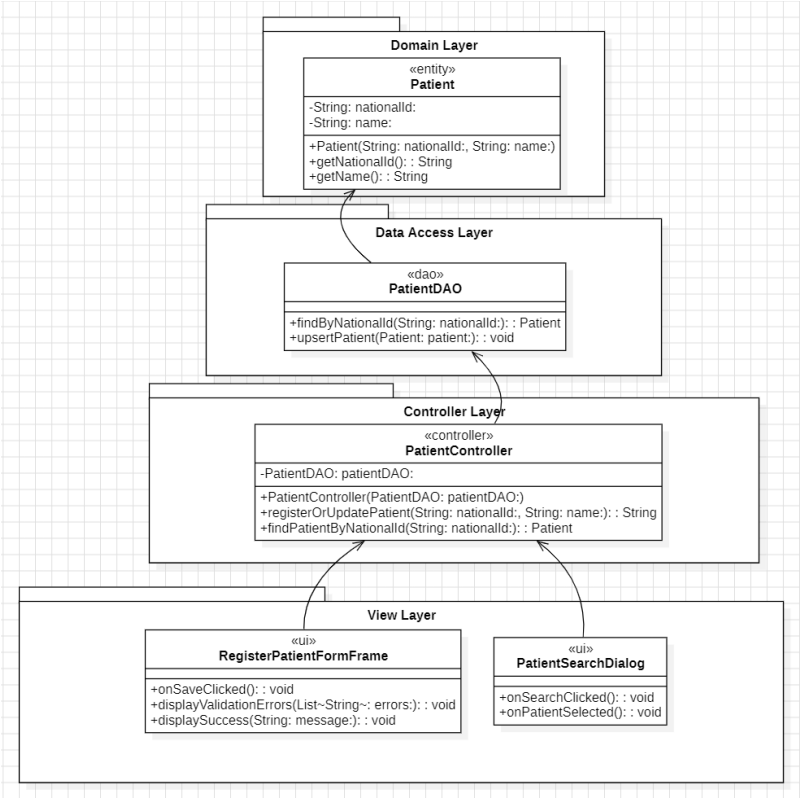
- Figure 1: UC-02 First-Cut Design Class Diagram



- Figure 2: UC-02 — CRC (Register patient information)



- Figure 3: UC-02 Updated Design Class Diagram.png



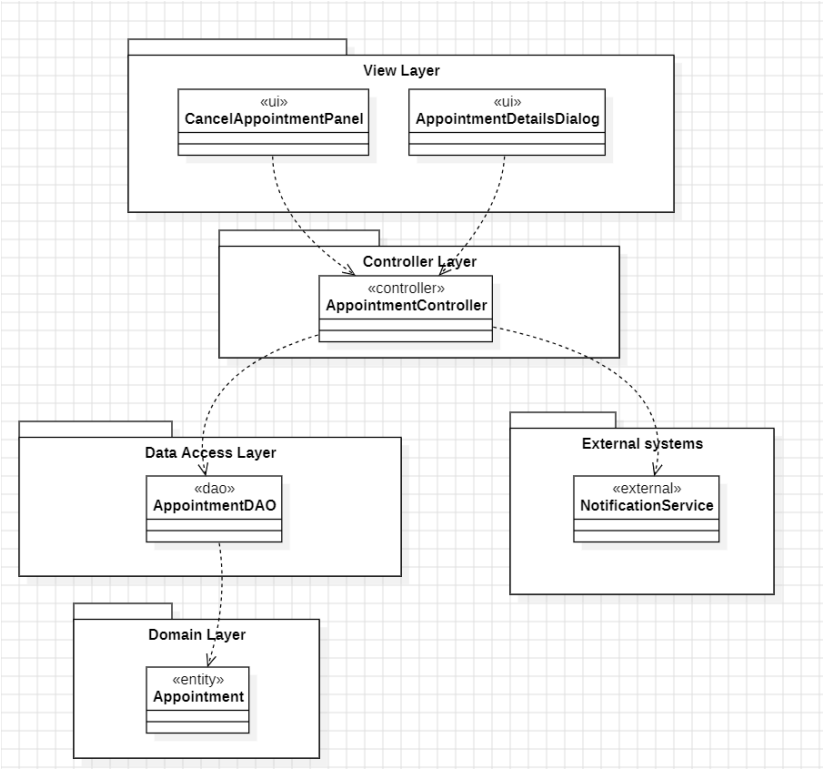
Extracted Operations Table (UC-02)

Step	Sender → Receiver	Assigned Class	Operation / Method Signature
1	UI → Controller	PatientController	registerPatient(nationalId, name)
2	Controller → DAO	PatientDAO	findByNationalId(nationalId)
3	Controller → DAO	PatientDAO	saveOrUpdate(patient)
4	Controller → UI	UI class	display success/error

6.2 UC-06 — Cancel Appointment (Simple — CRC)

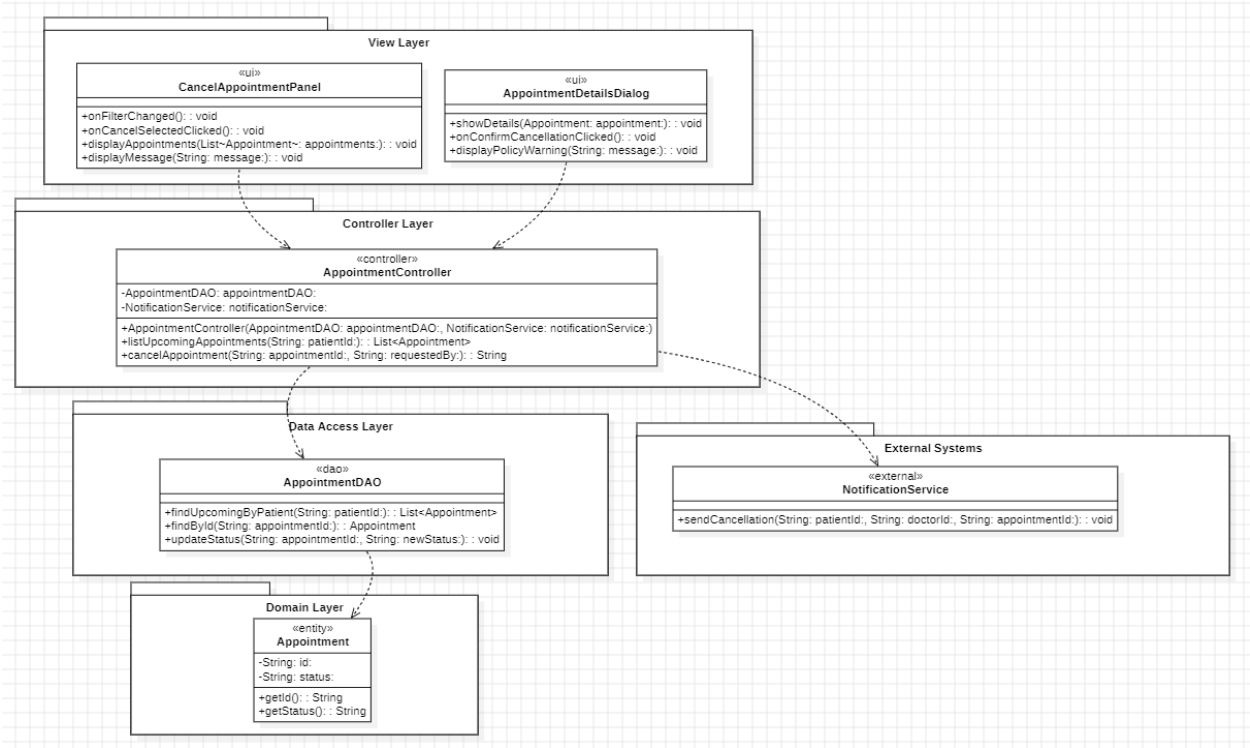
Figures

- Figure 4: UC-06 First-Cut Design Class Diagram (Partition).png



- Figure 5: UC-06 — CRC (Cancel appointment).png

- Figure 6: UC-06 Updated Design Class Diagram — AFTER CRC.png



Extracted Operations Table (UC-06)

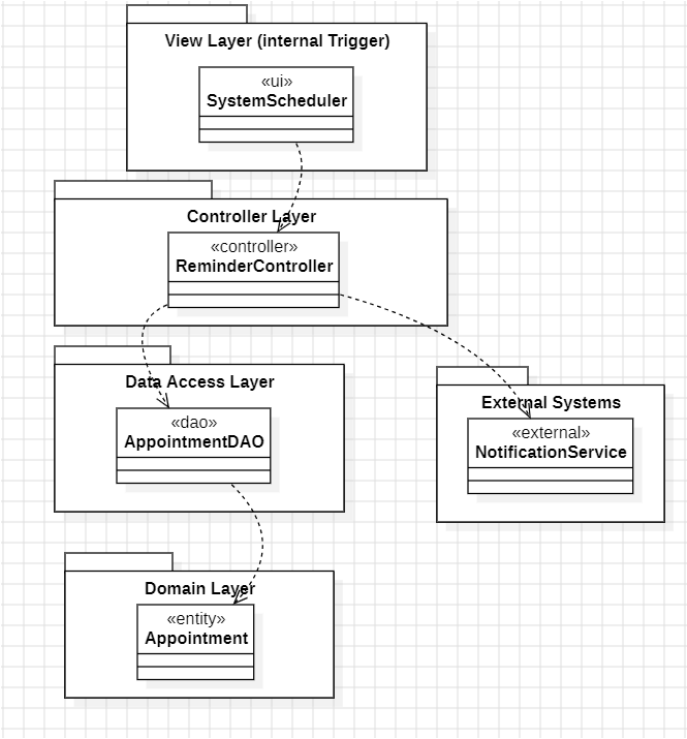
Step	Sender → Receiver	Assigned Class	Operation / Method Signature
1	UI → Controller	AppointmentController	listUpcomingAppointments(patientNationalId)
2	Controller → DAO	AppointmentDAO	findUpcomingByPatientNationalId(patientNationalId)
3	UI → Controller	AppointmentController	cancelAppointment(appointmentId)
4	Controller → DAO	AppointmentDAO	findById(appointmentId)
5	Controller → DAO	AppointmentDAO	updateStatus(appointmentId, "CANCELLED")

Step	Sender → Receiver	Assigned Class	Operation / Method Signature
6	Controller → External	NotificationService	sendCancellation(message)

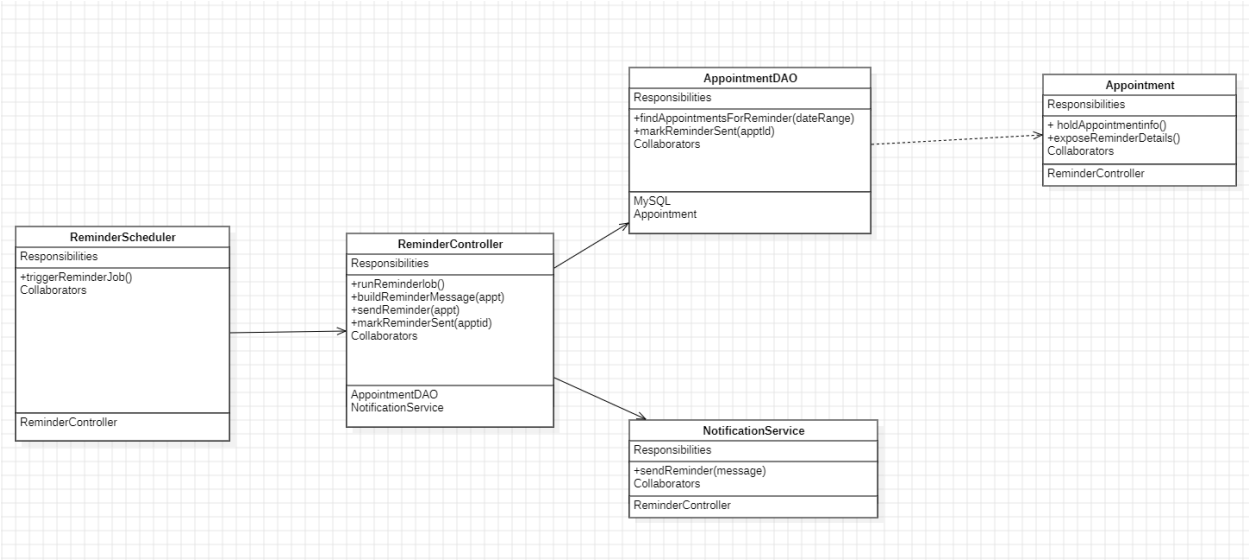
6.3 UC-09 — Send Appointment Reminders (Simple — CRC)

Figures

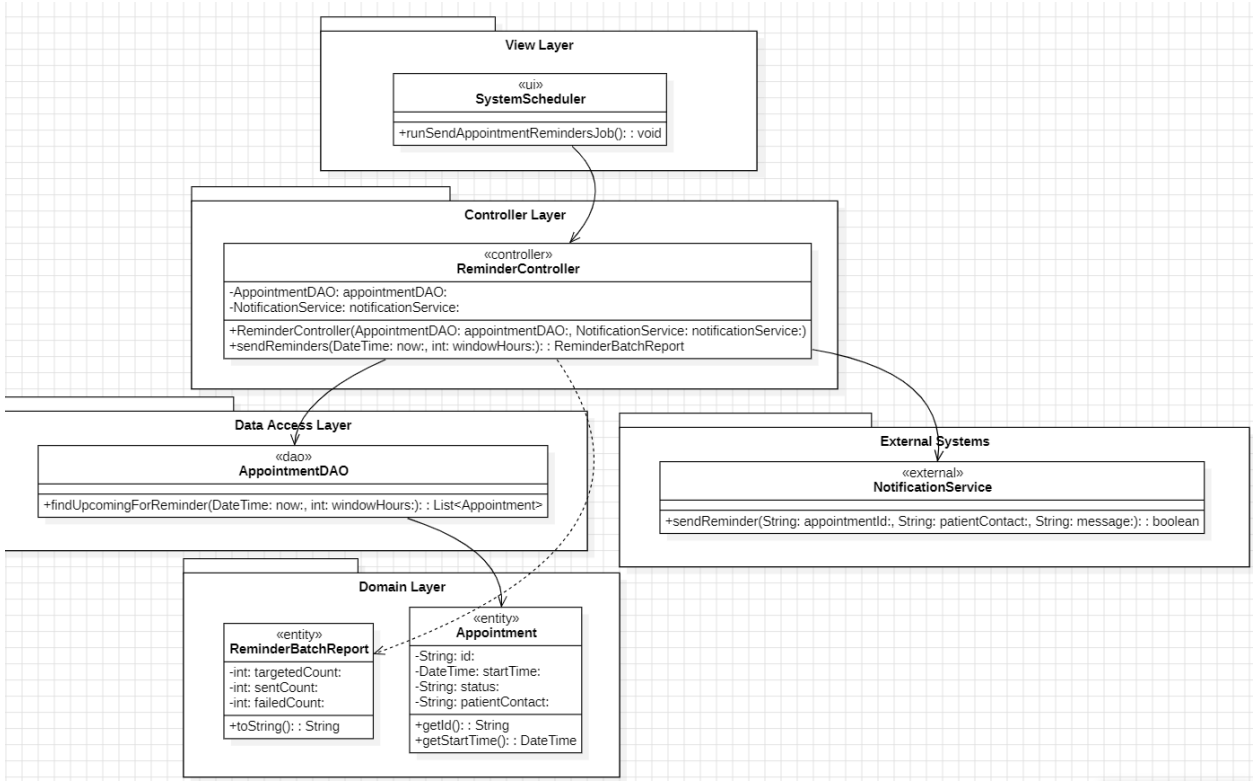
- Figure 7: UC-09 First-Cut Design Class Diagram (Partition).png



- Figure 8: UC-09 — CRC (Send appointment reminders).png



- Figure 9: UC-09 Updated Design Class Diagram — AFTER CRC.png



Extracted Operations Table (UC-09)

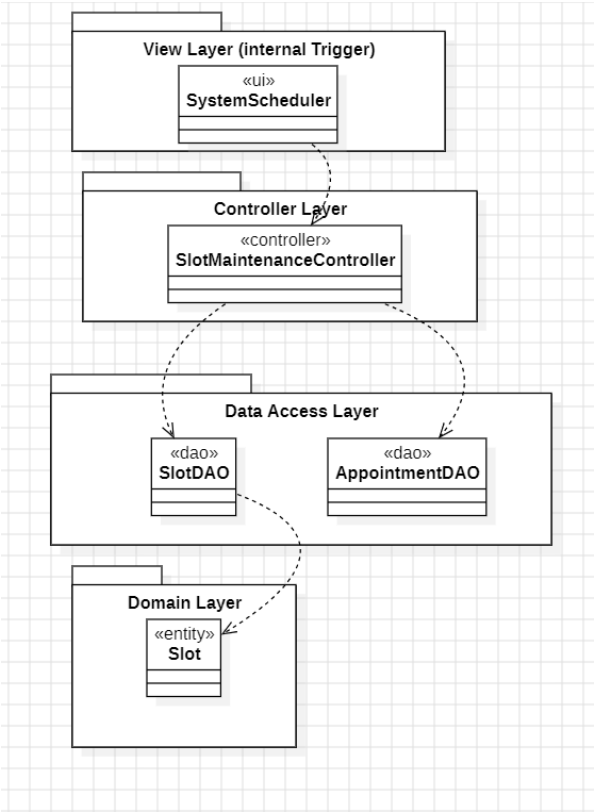
Step	Sender → Receiver	Assigned Class	Operation / Method Signature
1	Scheduler → Controller	ReminderController	sendReminders(..)

Step	Sender → Receiver	Assigned Class	Operation / Method Signature
2	Controller → DAO	AppointmentDAO	findAppointmentsForReminder(..)
3	Controller → External	NotificationService	send reminder message(s)

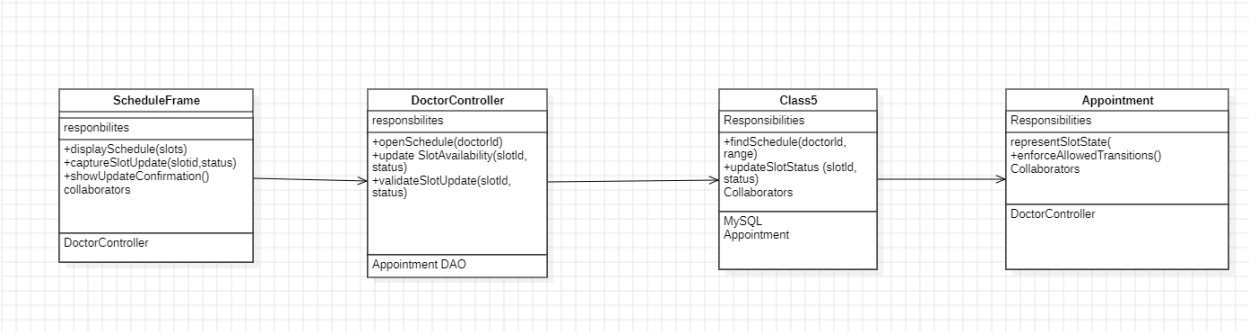
6.4 UC-T2 — Update Slot Availability (Simple — CRC)

Figures

- Figure 10: UC-T2 First-Cut Design Class Diagram (Partition).png

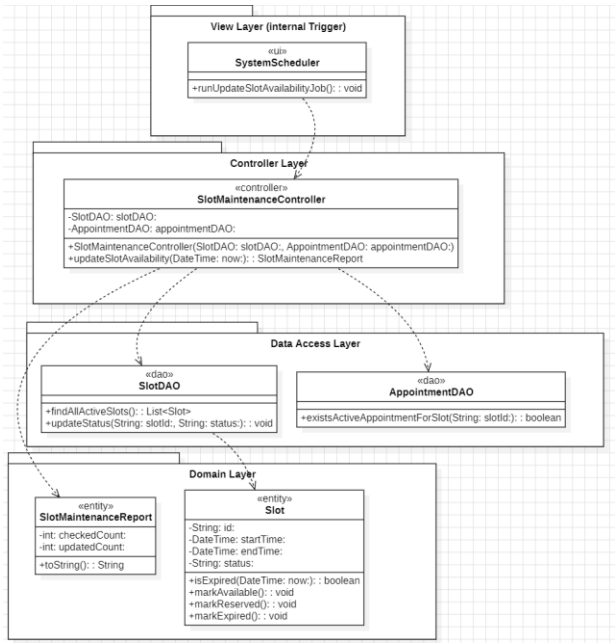


- Figure 11: UC-T2 — CRC (Update slot availability).png



-

- Figure 12: UC-T2 Updated Design Class Diagram — AFTER CRC.png



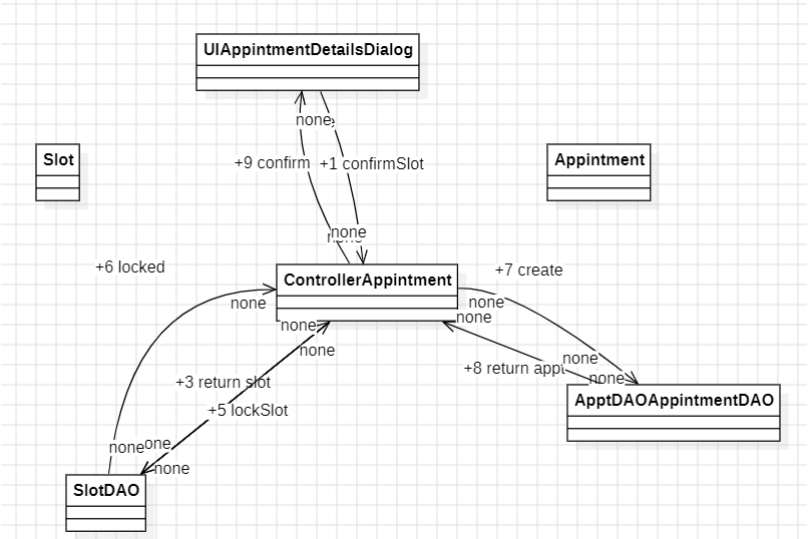
Extracted Operations Table (UC-T2)

Step	Sender → Receiver	Assigned Class	Operation / Method Signature
1	Scheduler → Controller	SlotMaintenanceController	updateSlotAvailability(..)
2	Controller → DAO	SlotDAO	findAllActiveSlots()
3	Controller → DAO	SlotDAO	updateStatus(slotId,status)

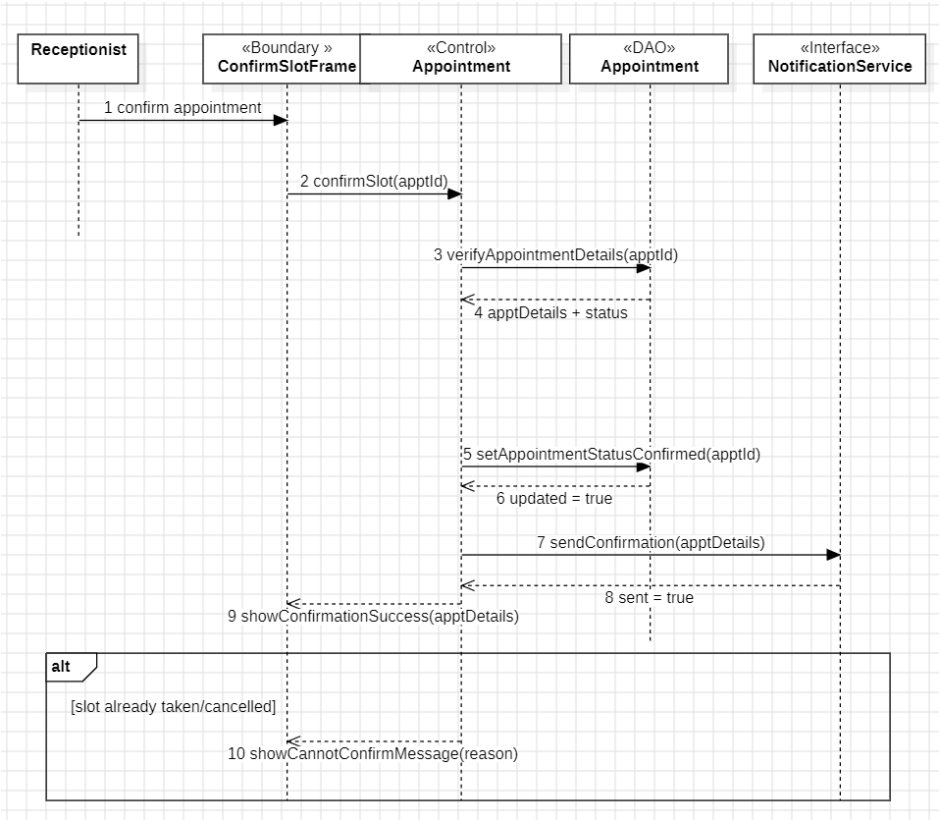
6.5 UC-04 — Confirm Appointment Slot (Moderate — Communication)

Figures

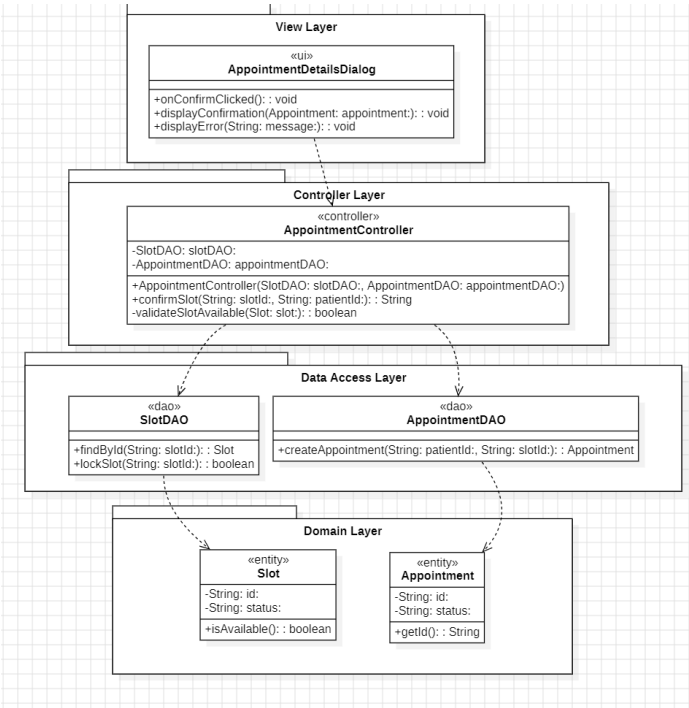
- Figure 13: UC-04 First-Cut Design Class Diagram (Partition).png



- Figure 14: UC-04 Confirm appointment slot.png



- Figure 15: UC-04 Updated Design Class Diagram — AFTER communication diagram.png



Extracted Operations Table (UC-04)

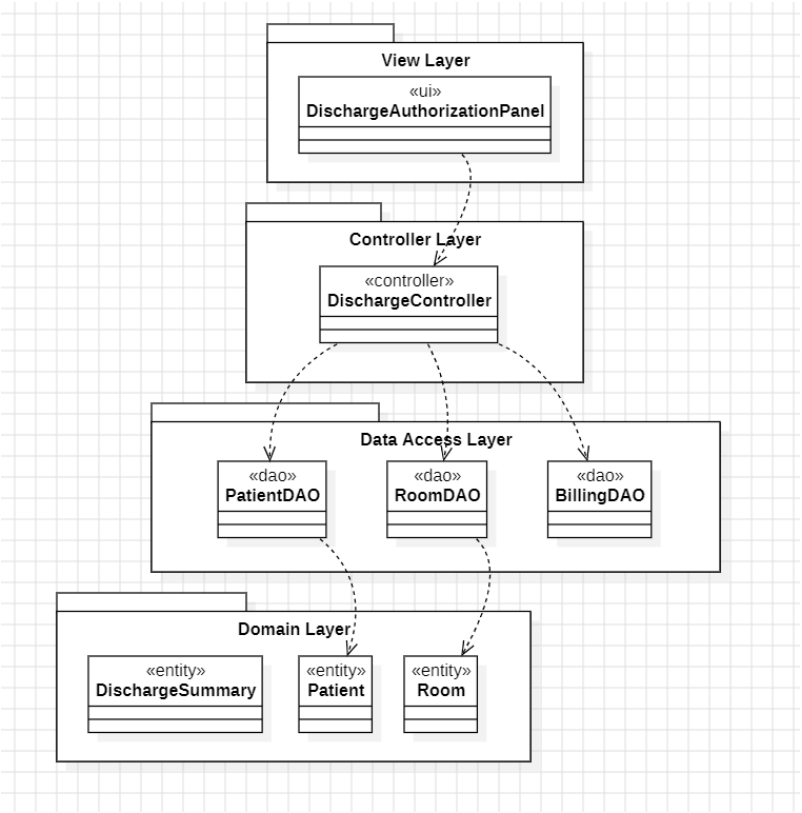
Message	Sender → Receiver	Assigned Class	Operation / Method Signature
1	UI → Controller	AppointmentController	confirmSlot(appointmentId)
2	Controller → DAO	AppointmentDAO	verify + update status confirmed
3	Controller → External	NotificationService	send confirmation

---

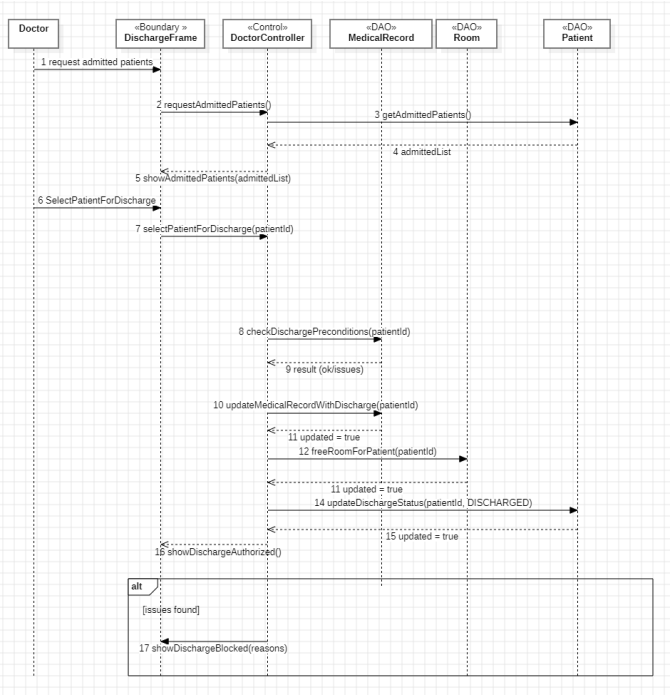
6.6 UC-08 — Authorize Patient Discharge (Moderate — Communication)

Figures

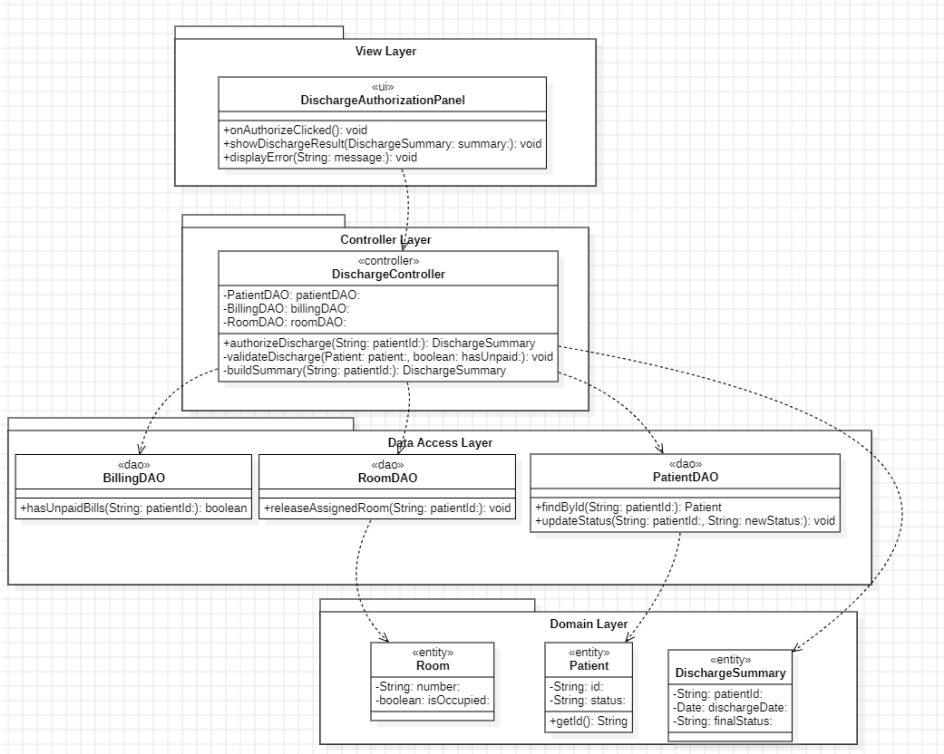
- Figure 16: UC-08 Multi-Layer Communication Diagram.png



- Figure 17: UC-08 Authorize patient discharge.png



- Figure 18: UC-08 Updated Design Class Diagram — AFTER communication diagram.png



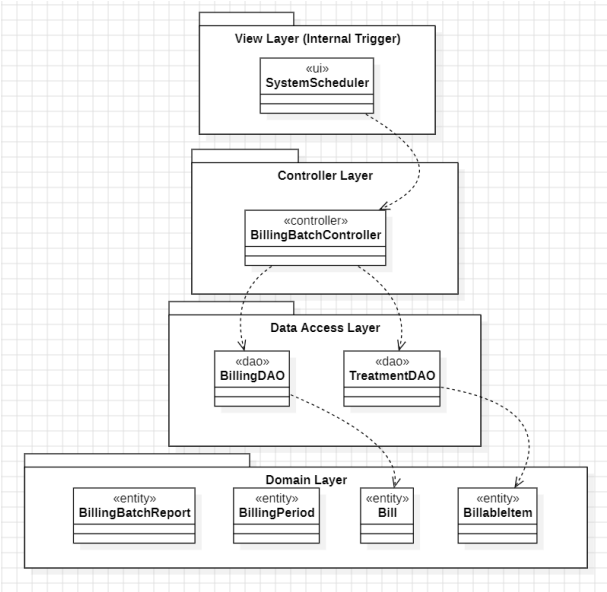
Extracted Operations Table (UC-08)

Message	Sender → Receiver	Assigned Class	Operation / Method Signature
1	UI → Controller	DischargeController	authorizeDischarge(patientId)
2	Controller → DAO	MedicalRecordDAO	checkDischargePreconditions(patientId)
3	Controller → DAO	MedicalRecordDAO	updateMedicalRecordWithDischarge(patientId)
4	Controller → DAO	RoomDAO	freeRoomForPatient(patientId)
5	Controller → DAO	PatientDAO	updateDischargeStatus(patientId,status)

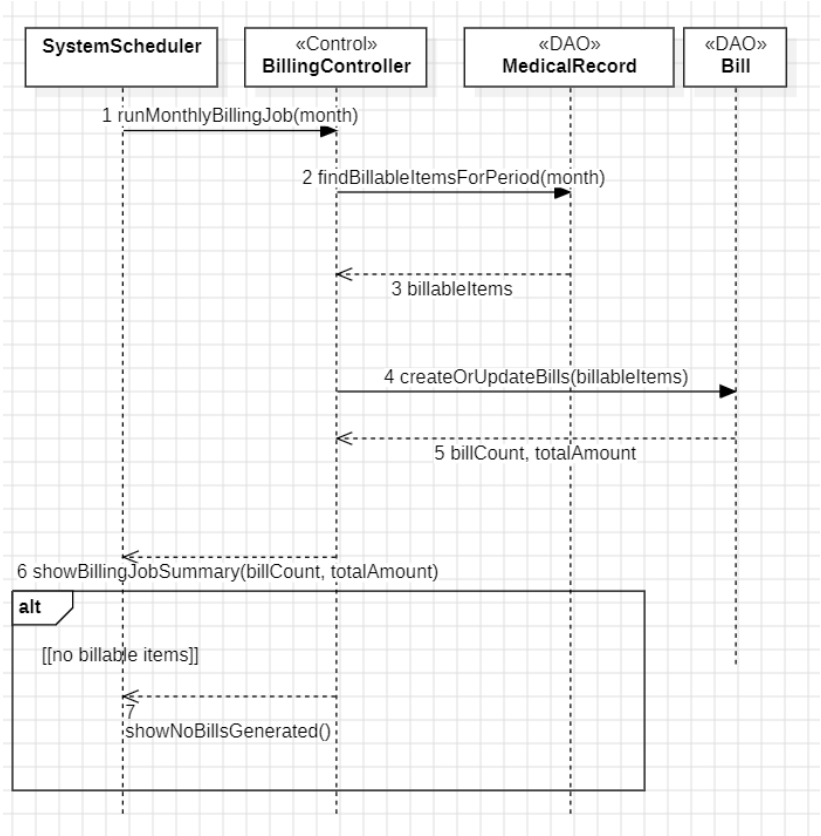
6.7 UC-11 — Generate Monthly Bill (Moderate — Communication)

Figures

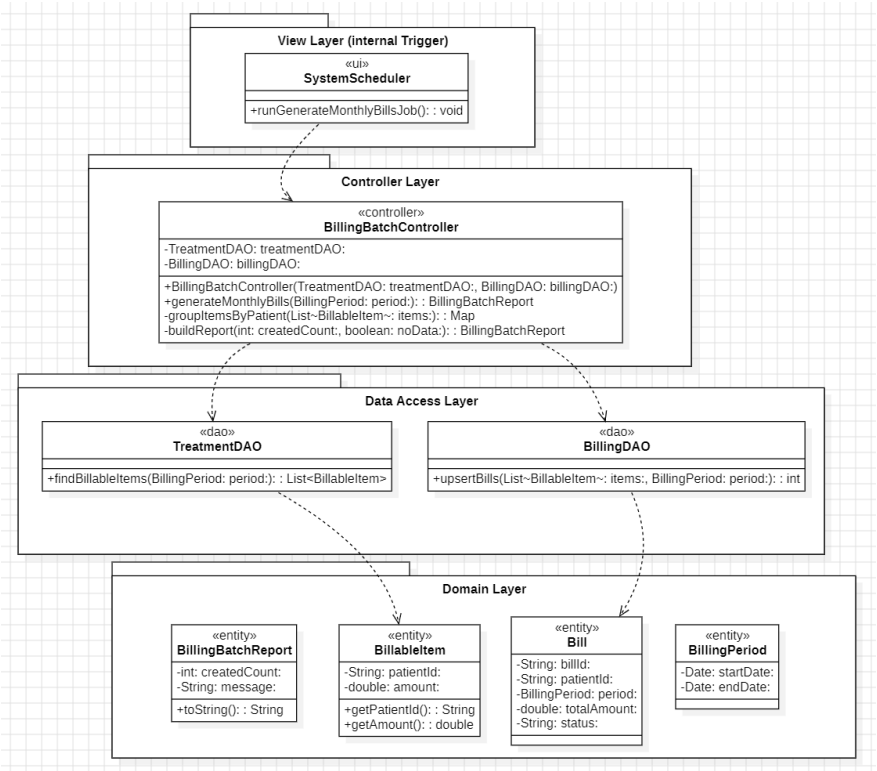
- Figure 19: UC-11 Updated Design Class Diagram (Partition).png



- Figure 20: UC-11 Generate Monthly Bill — Communication.png



- Figure 21: UC-11 Updated Design Class Diagram — AFTER communication diagram.png



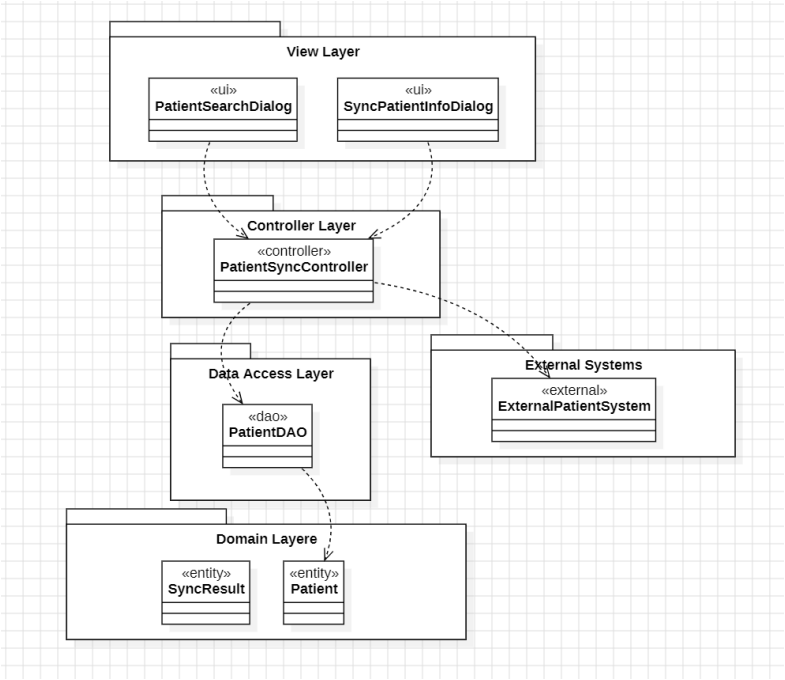
Extracted Operations Table (UC-11)

Message	Sender → Receiver	Assigned Class	Operation / Method Signature
1	Scheduler → Controller	BillingController	generateMonthlyBills(period)
2	Controller → DAO(s)	DAO(s)	retrieve billable items
3	Controller → Domain	Bill	calculate totals
4	Controller → DAO	BillDAO	create/update bills
Alt	Controller	BillingController	handle “no bills generated”

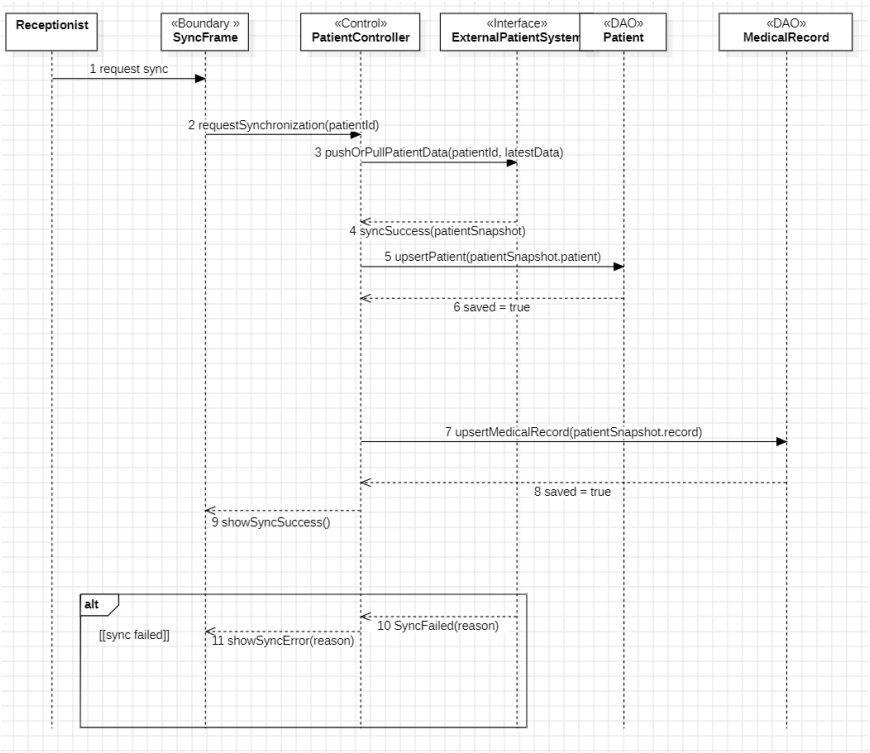
6.8 UC-T3 — Synchronize Patient Information (Moderate — Communication)

Figures

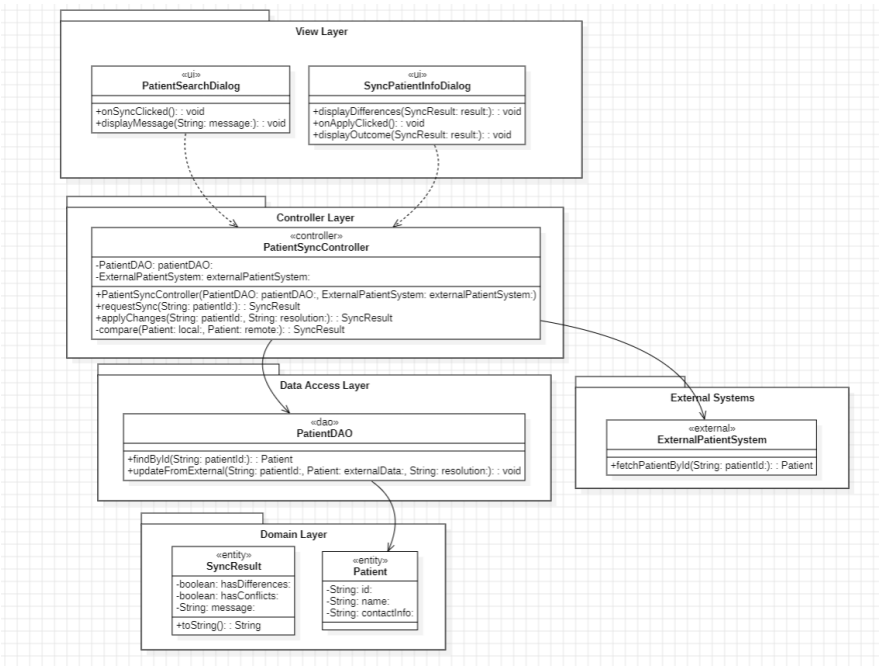
- Figure 22: UC-T3 First-Cut Design Class Diagram (Partition).png



- Figure 23: UC-T3 Synchronize patient information.png



- Figure 24: UC-T3 Updated Design Class Diagram — AFTER communication diagram.png



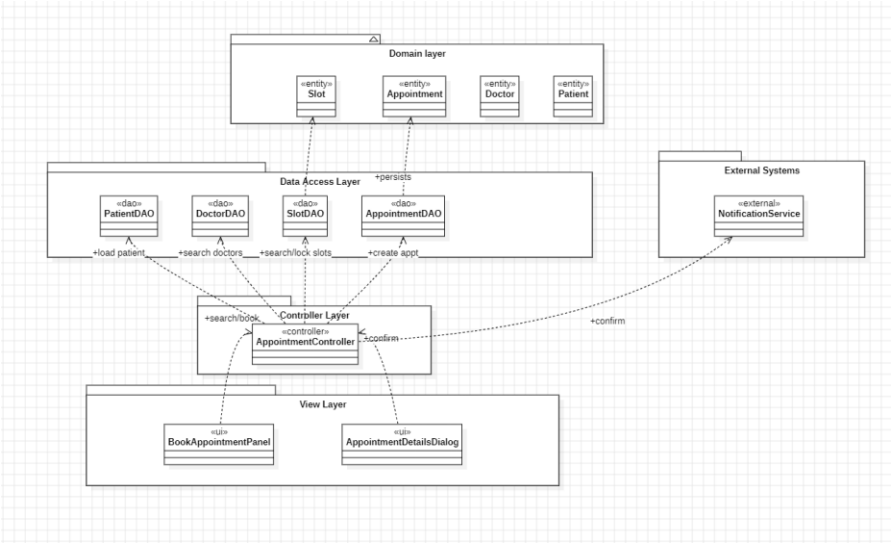
Extracted Operations Table (UC-T3)

Message	Sender → Receiver	Assigned Class	Operation / Method Signature
1	UI → Controller	PatientController	start synchronization
2	Controller → External	ExternalPatientSystem	fetch patient snapshot
3	Controller → DAO	PatientDAO	upsert patient
4	Controller → DAO	MedicalRecordDAO	upsert record
Alt	Controller → UI	UI class	show sync error

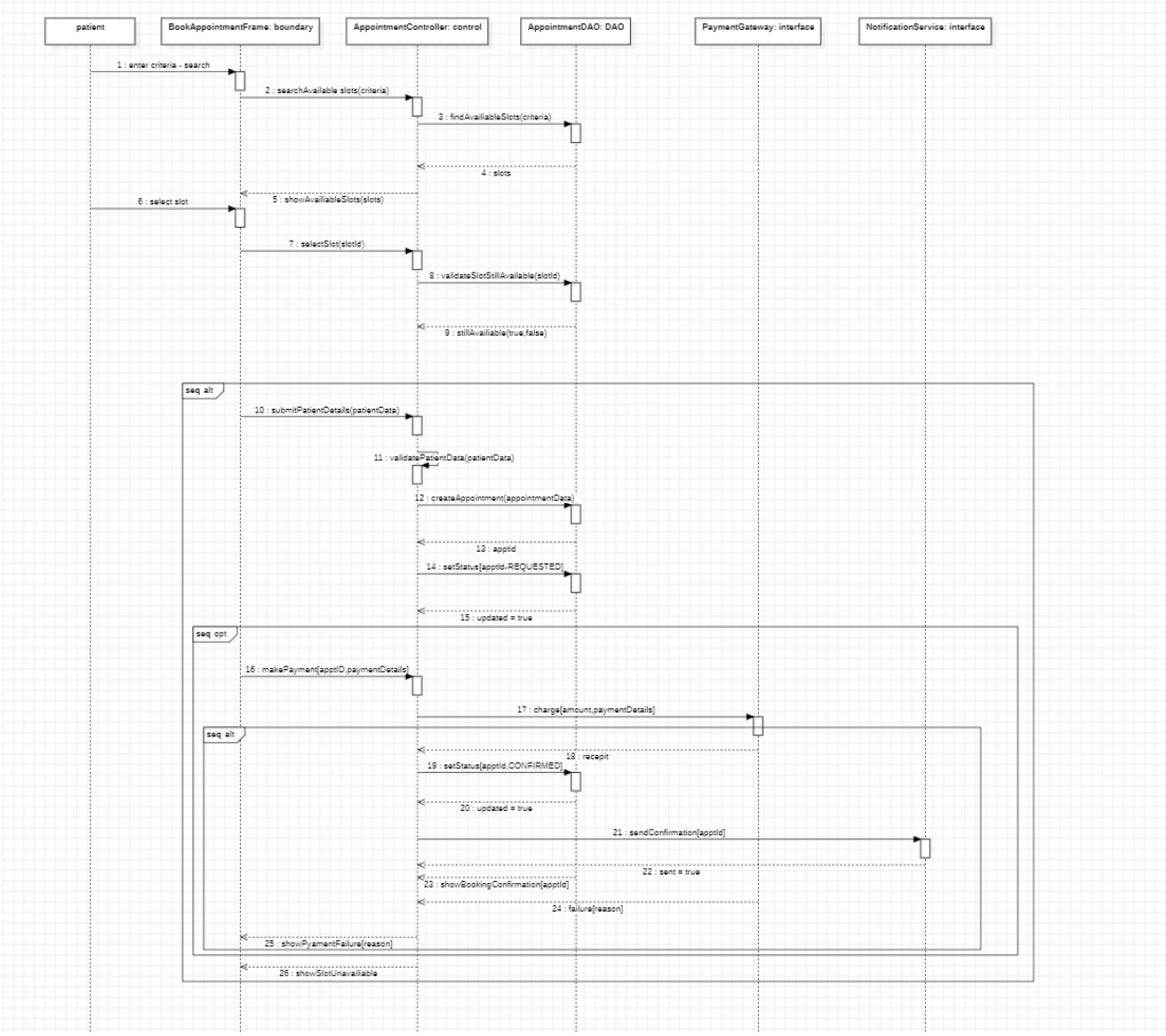
6.9 UC-01 — Book Appointment (Complex — Sequence)

Figures

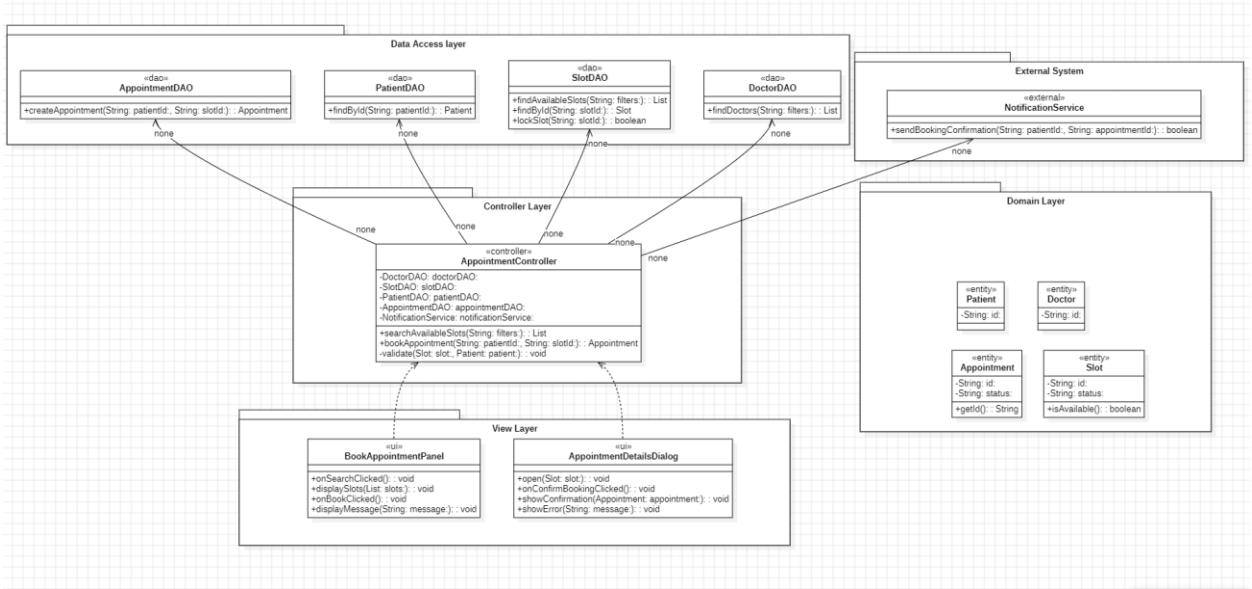
- Figure 25: UC-01 First-Cut Design Class Diagram (Partition).png



- Figure 26: UC-01 Book appointment.png



- Figure 27: UC-01 First-Cut Design Class Diagram — AFTER sequence diagram.png



Extracted Operations Table (UC-01)

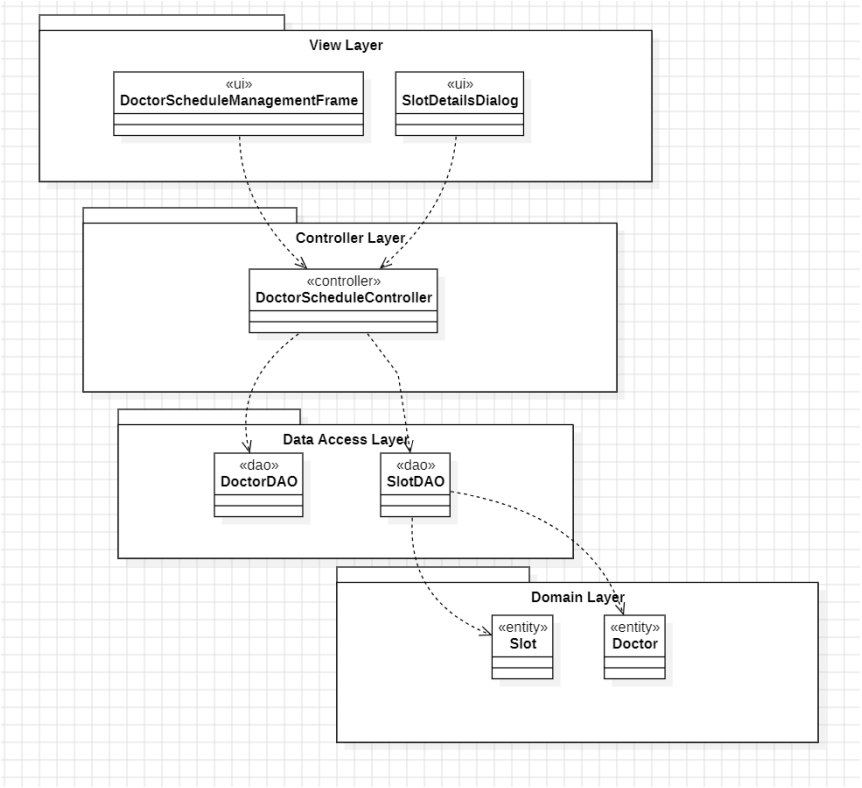
Step	Sender → Receiver	Assigned Class	Operation / Method Signature
1	UI → Controller	AppointmentController	search/select slot
2	Controller → DAO	DAO(s)	validate availability + create appointment
3	Controller → External	NotificationService	send confirmation

---

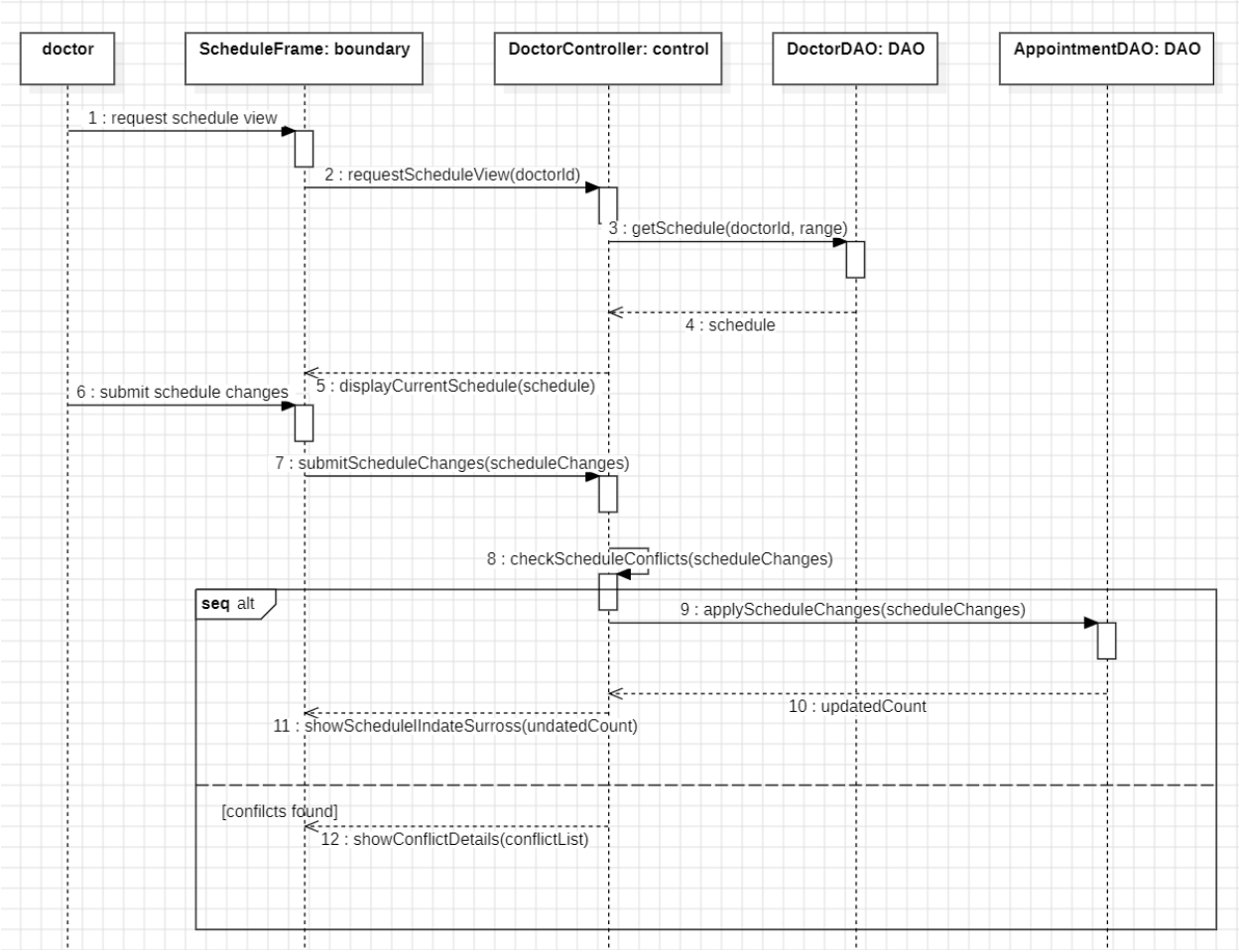
6.10 UC-03 — Manage Doctor Schedule (Complex — Sequence)

Figures

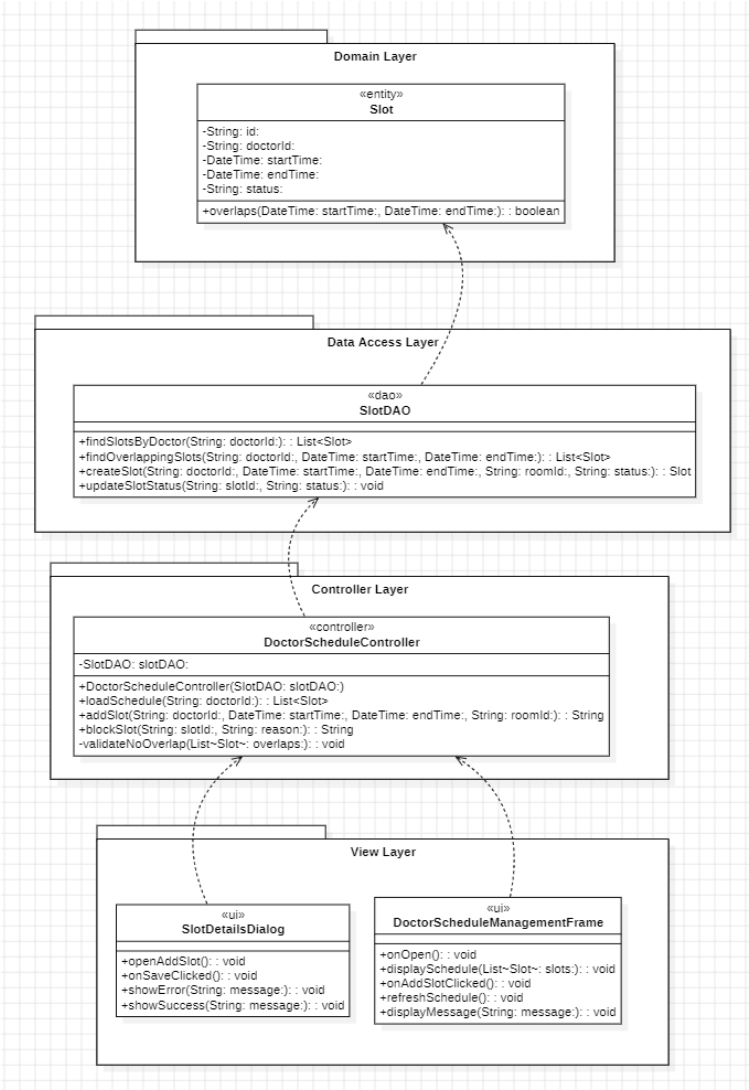
- Figure 28: UC-03 First-Cut Design Class Diagram (Partition).png



- Figure 29: UC-03 Manage doctor schedule.png



- Figure 30: UC-03 First-Cut Design Class Diagram — AFTER sequence diagram.png



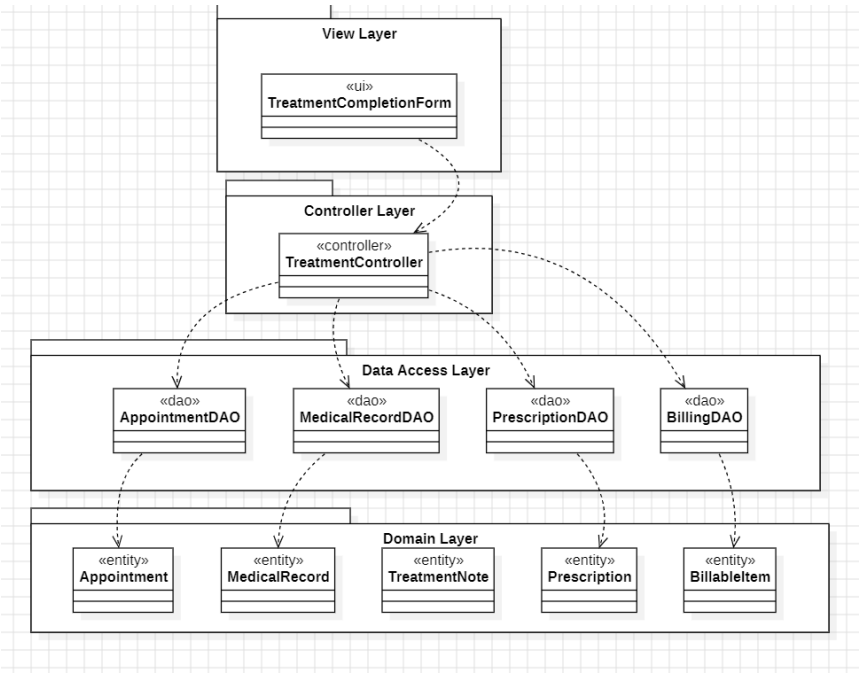
Extracted Operations Table (UC-03)

Step	Sender → Receiver	Assigned Class	Operation / Method Signature
1	UI → Controller	DoctorScheduleController	open schedule
2	Controller → DAO	DAO(s)	load schedule
3	UI → Controller	DoctorScheduleController	propose changes
4	Controller → Controller	DoctorScheduleController	detect conflicts
5	Controller → DAO	SlotDAO	persist changes

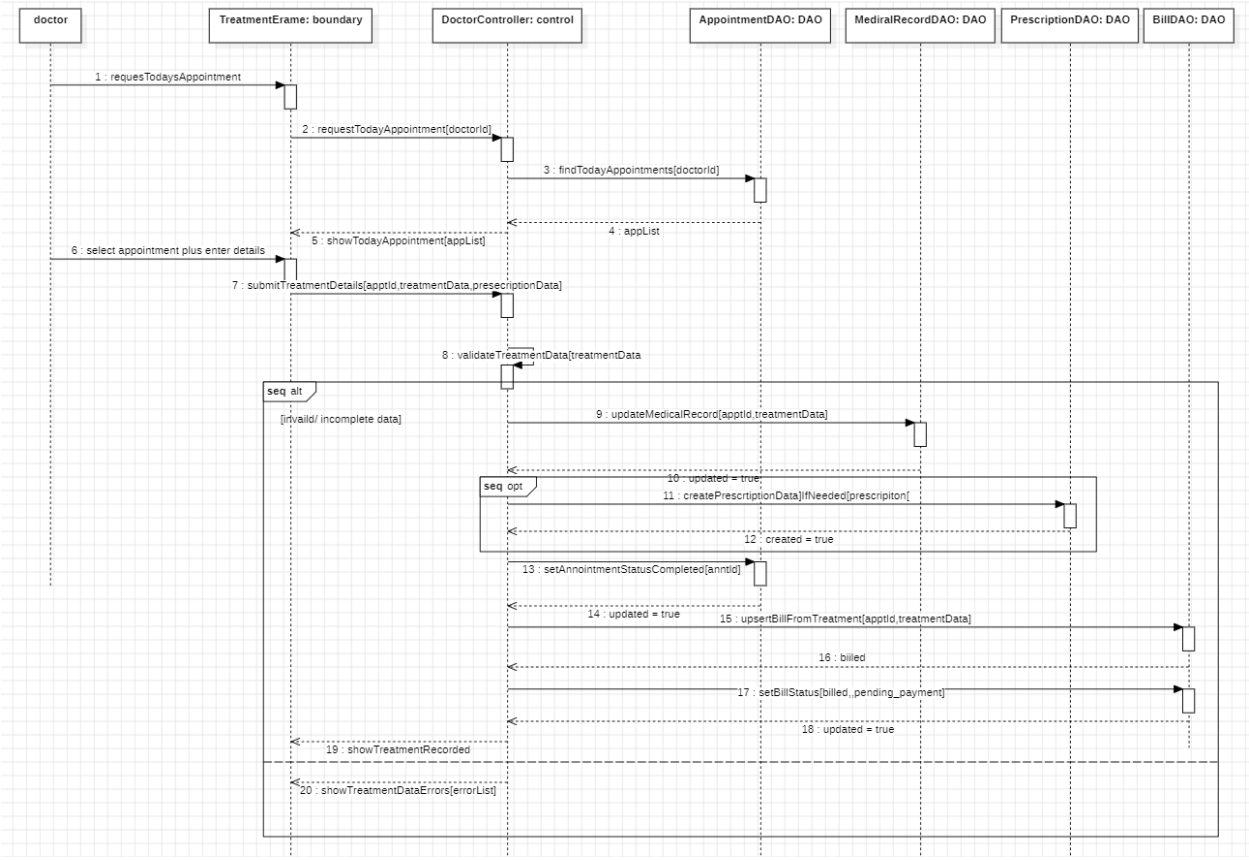
6.11 UC-05 — Complete Patient Treatments (Complex — Sequence)

Figures

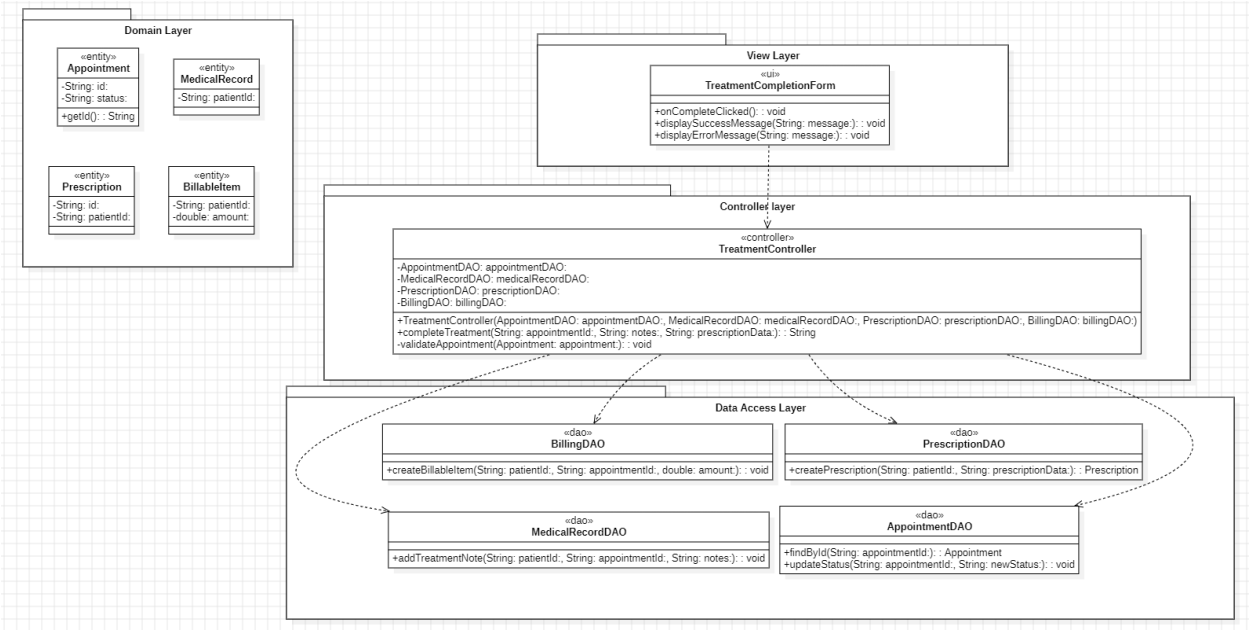
- Figure 31: UC-05 First-Cut Design Class Diagram (Partition).png



- Figure 32: UC-05 Complete patient treatments.png



- Figure 33: UC-05 First-Cut Design Class Diagram — AFTER sequence diagram.png



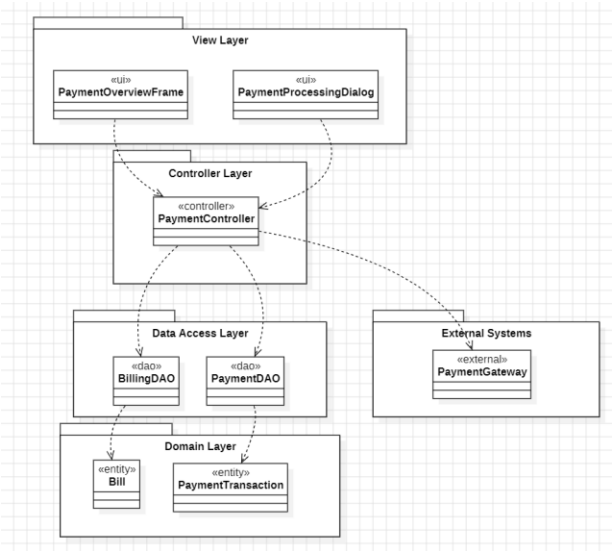
Extracted Operations Table (UC-05)

Step	Sender → Receiver	Assigned Class	Operation / Method Signature
1	UI → Controller	TreatmentController	complete treatment
2	Controller → DAO	MedicalRecordDAO	update record
3	Controller → DAO	PrescriptionDAO	optional prescription
4	Controller → DAO	BillDAO	trigger billing

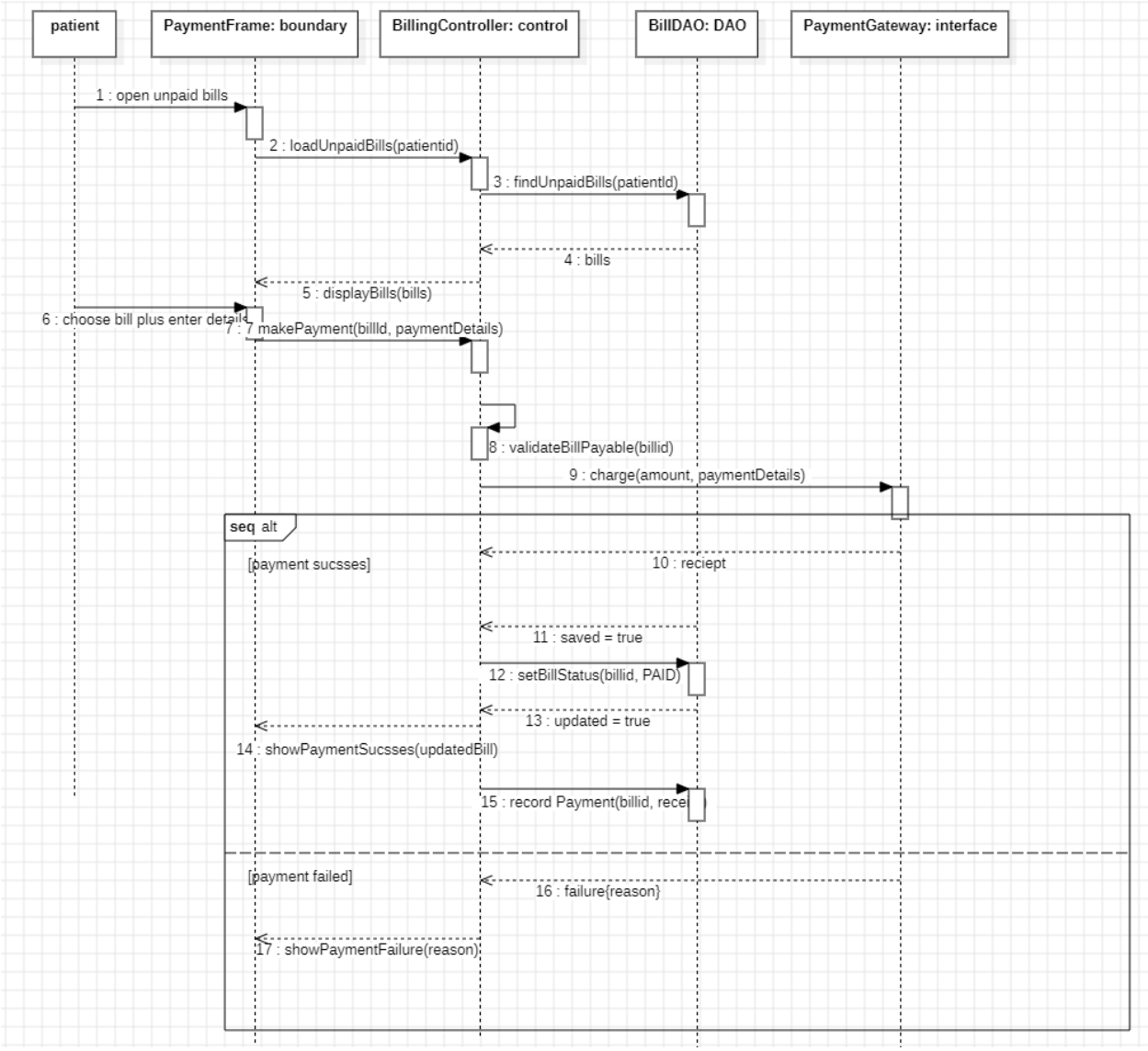
6.12 UC-07 — Make Payment (Complex — Sequence)

Figures

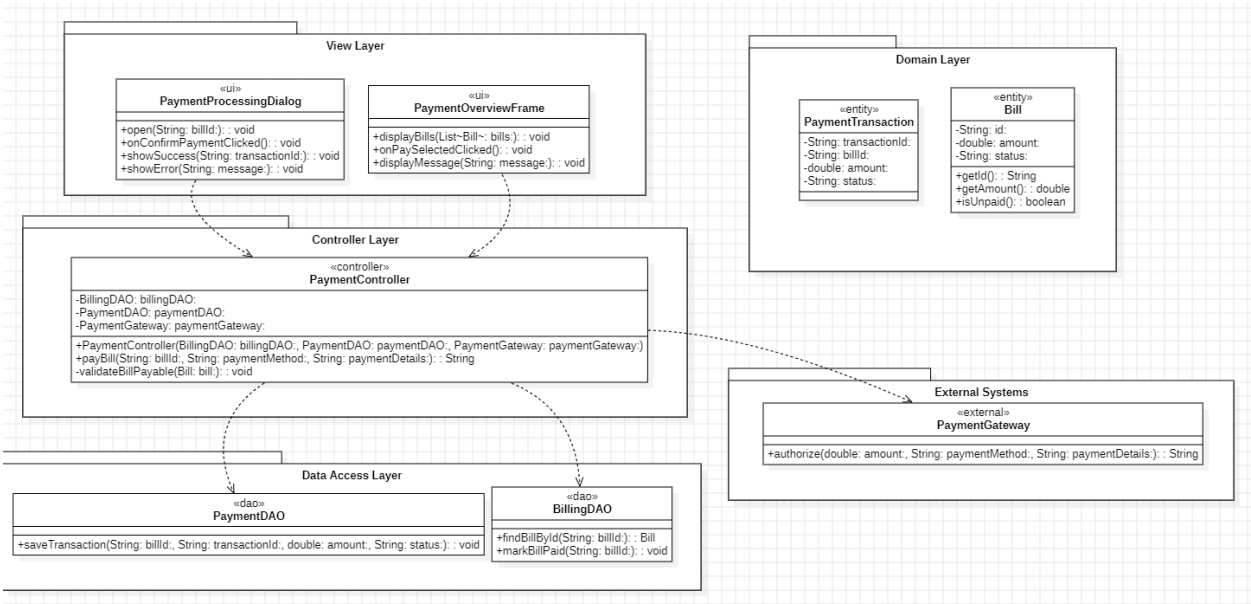
- Figure 34: UC-07 First-Cut Design Class Diagram (Partition).png



- Figure 35: UC-07 Make payment.png



- Figure 36: UC-07 First-Cut Design Class Diagram — AFTER sequence diagram.png



Extracted Operations Table (UC-07)

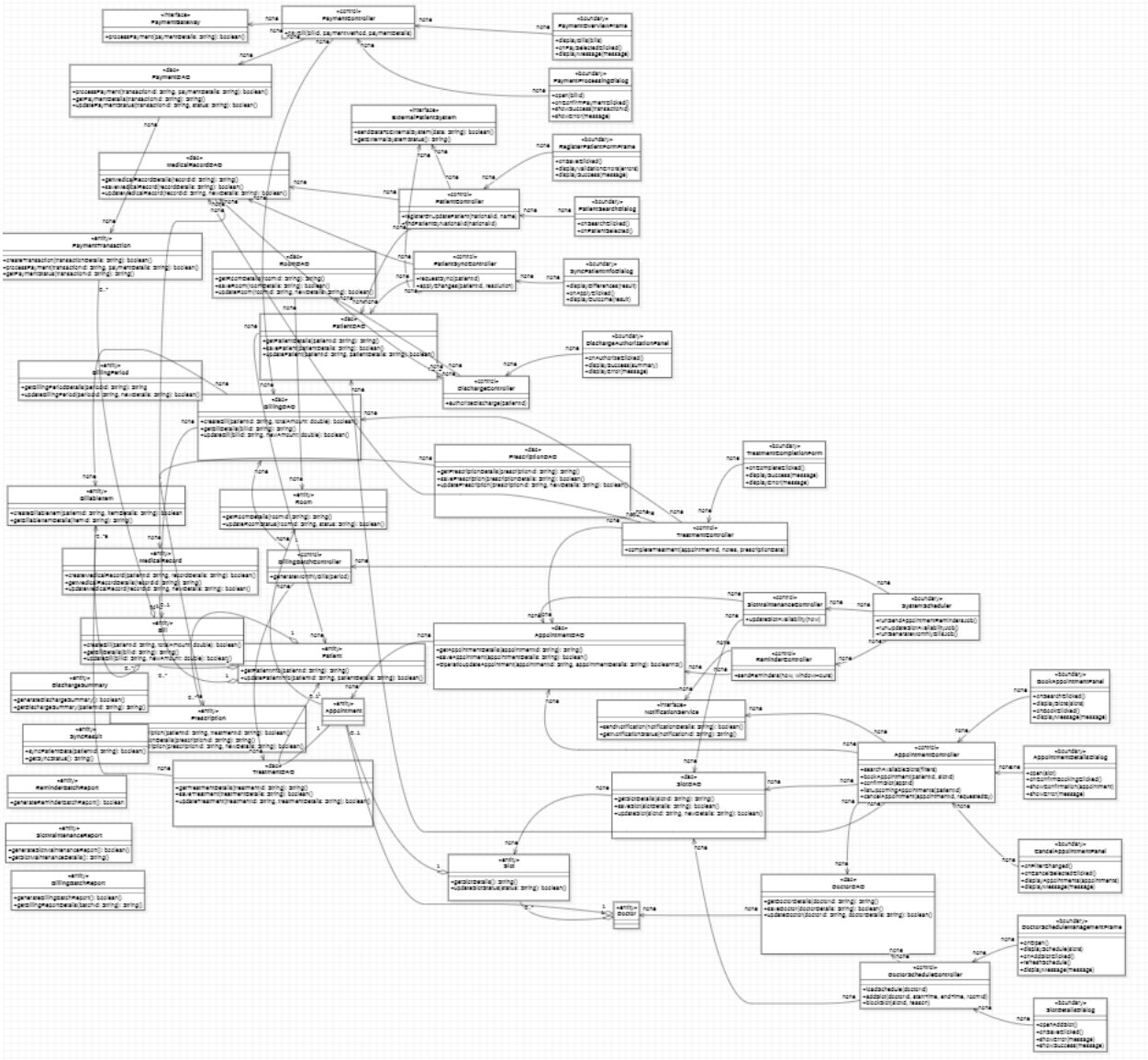
Step	Sender → Receiver	Assigned Class	Operation / Method Signature
1	UI → Controller	PaymentController	make payment
2	Controller → DAO	BillDAO	load unpaid bills
3	Controller → External	PaymentGateway	charge payment
4	Controller → DAO	BillDAO	record payment + update status

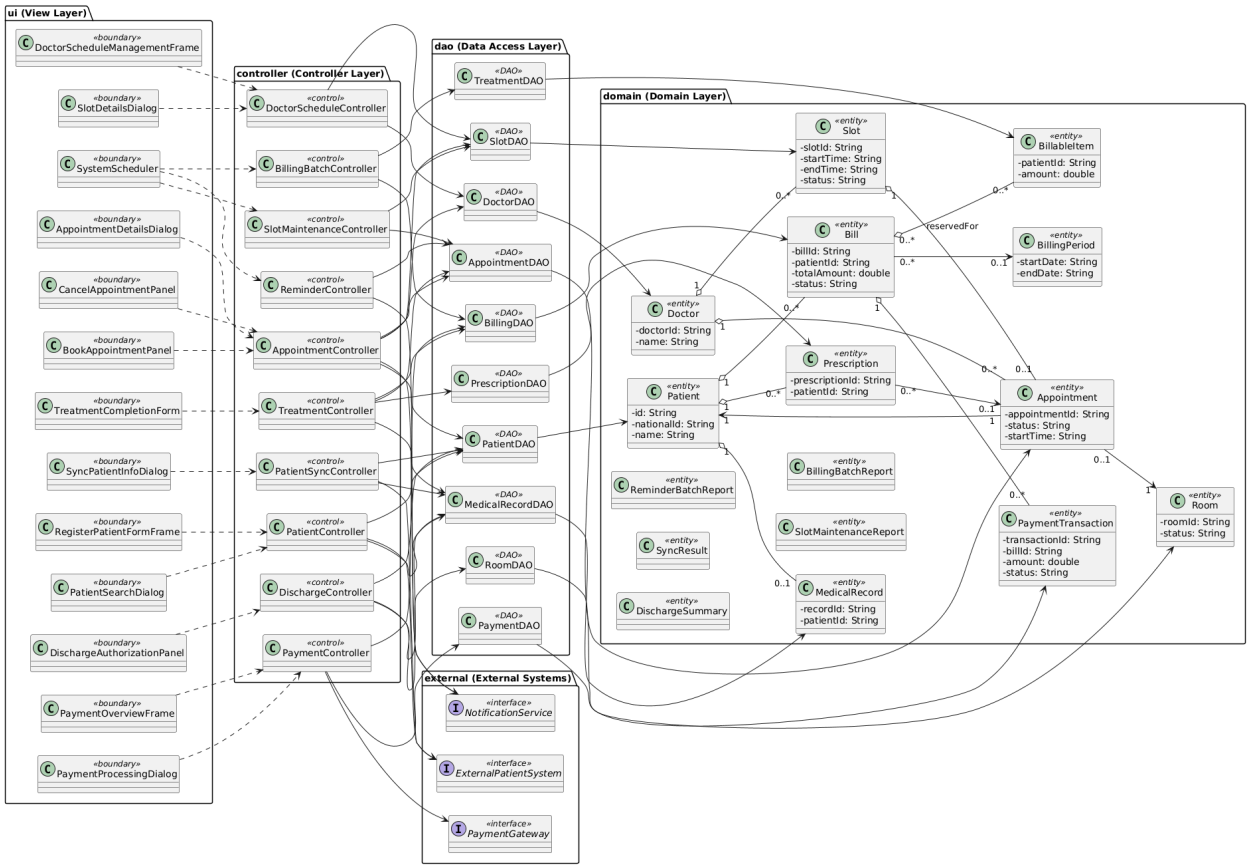
---

7. Final Consolidated Design Class Diagram (DCD)

Figure

- Figure 37: Final Design Class Diagram.png





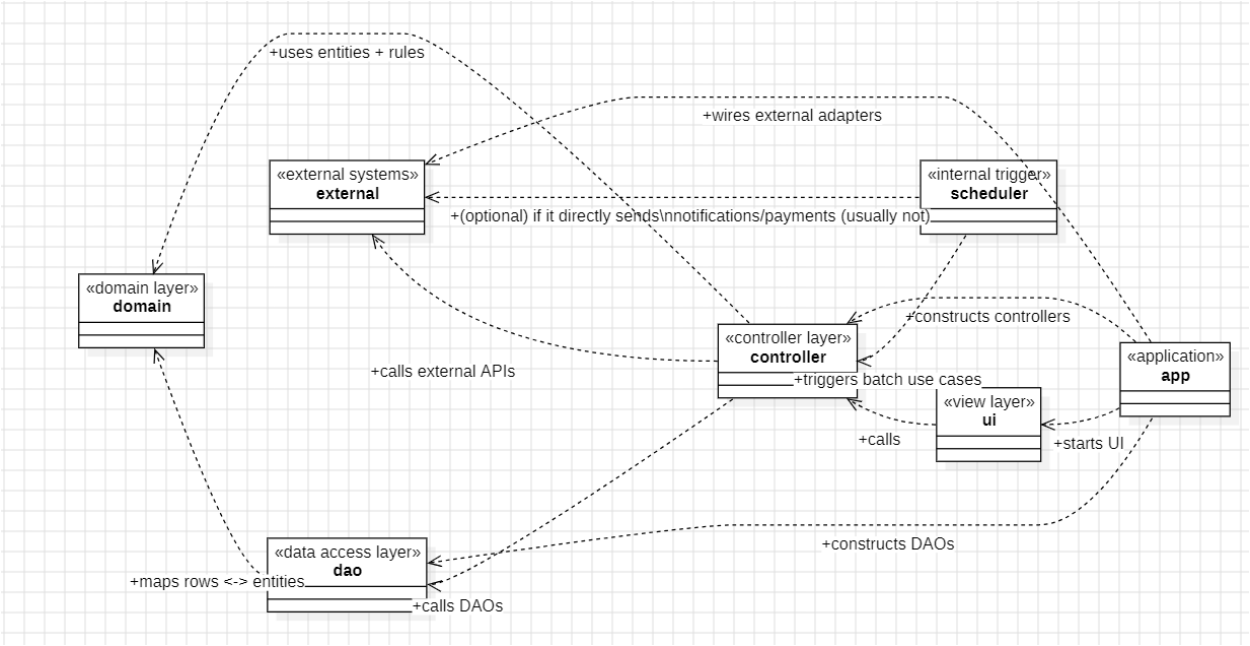
The final DCD consolidates all discovered classes and operations from the 12 realizations and enforces layered responsibilities.

---

8. Package Diagram (Architecture)

Figure

- Figure 38: Package Diagram.png



The package diagram documents the final architecture dependency direction:

- ui → controller
- controller → domain, dao, external
- dao → domain
- domain has no outgoing dependencies

Forbidden dependencies are avoided (ui→dao, domain→dao).

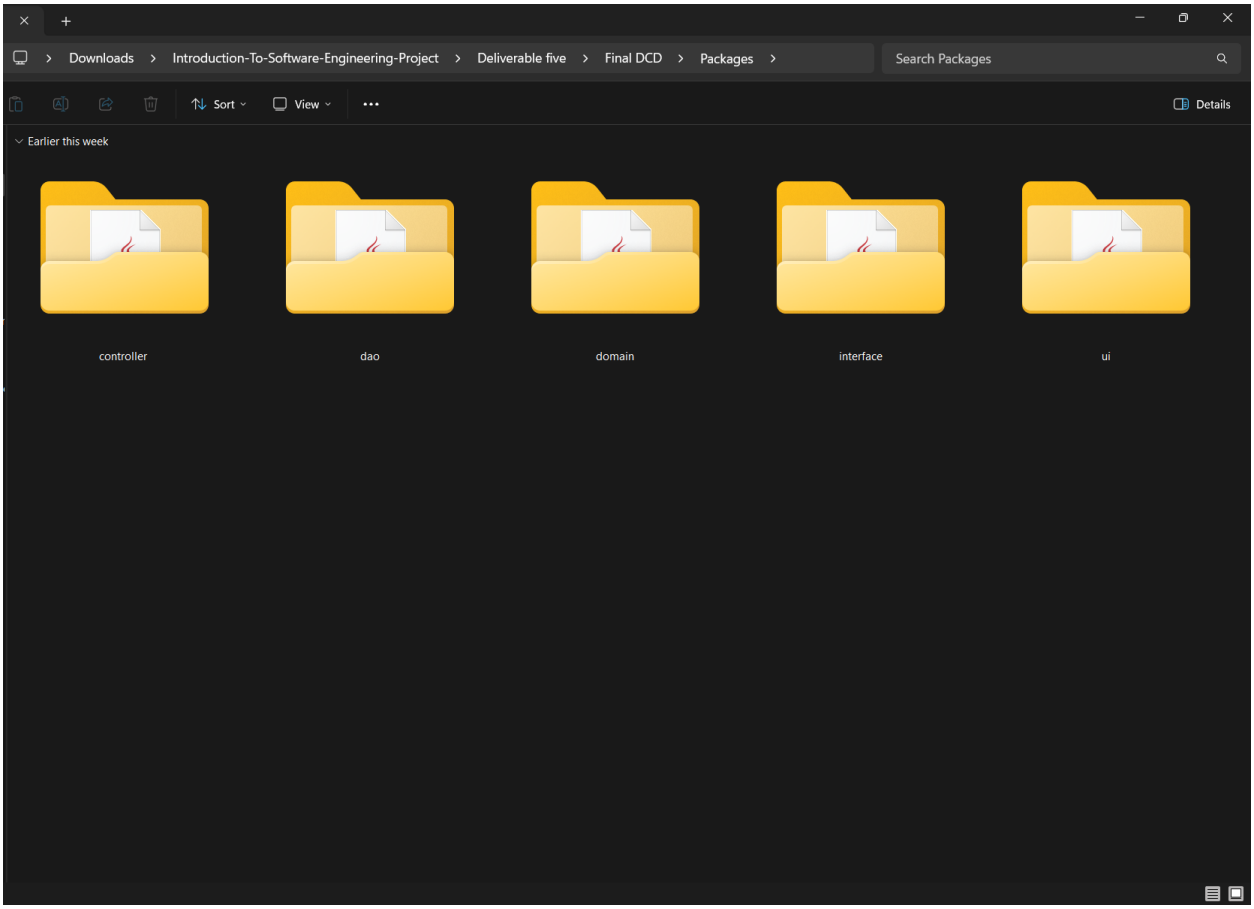
---

9. Implementation (StarUML → Java + Walking Skeleton, Arrays Only)

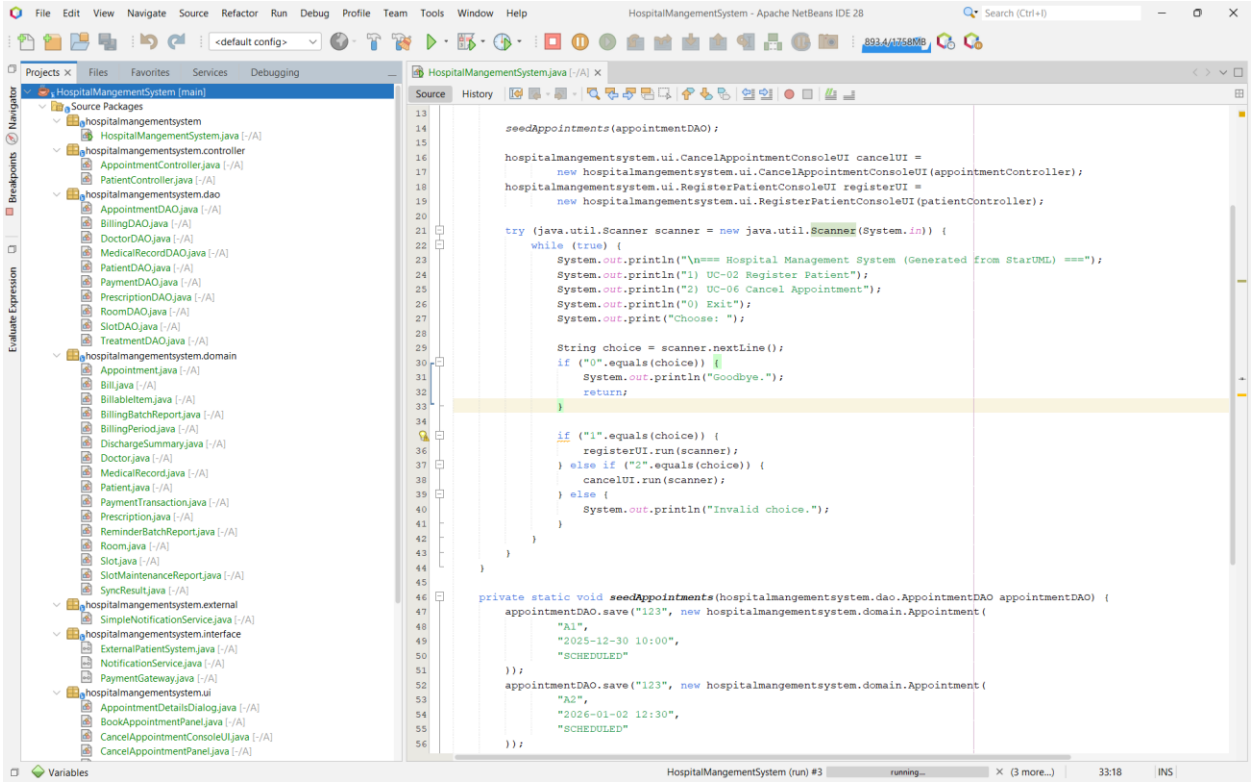
9.1 Model-to-Code Transformation

We generated Java skeleton code from StarUML and refined it into runnable code in NetBeans while maintaining UML consistency.

- Figure 39:



- Figure 40: (NetBeans package tree)

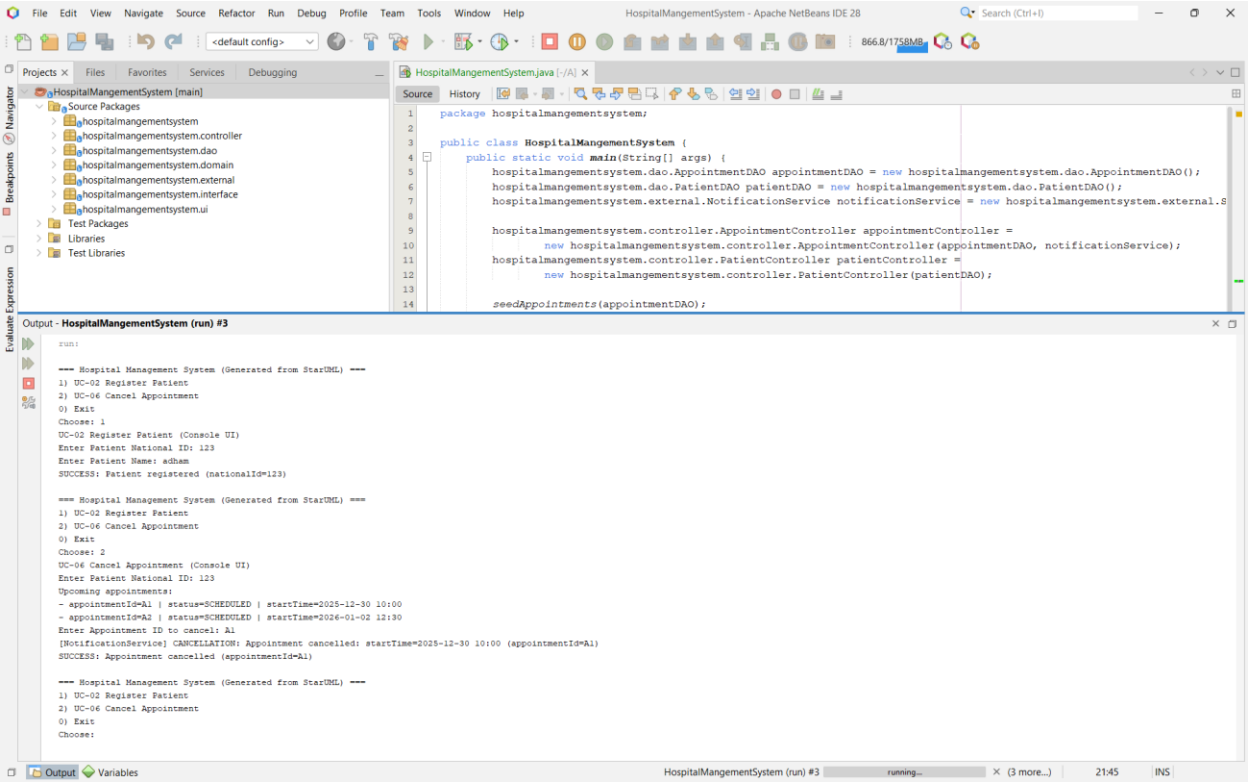


9.2 Implemented Use Cases

UC-02 Register Patient Information (Implemented)

- UI collects nationalId and name
- Controller validates and registers patient
- DAO persists using arrays

Figure 41: (Running UC-02 success output)



UC-06 Cancel Appointment (Implemented)

- UI lists upcoming appointments
- Controller cancels selected appointment
- DAO updates status to CANCELLED
- Notification service sends cancellation message

9.3 “Database Interaction” via Array-Based DAOs

Persistence requirement is satisfied through arrays inside PatientDAO and AppointmentDAO.

- Figure 44: (Code snippet of PatientDAO array storage)

HospitalMangementSystem.java [-/A] × PatientDAO.java [-/A] ×

Source History

```java
package hospitalmangementsystem.dao;

import hospitalmangementsystem.domain.Patient;

public class PatientDAO {
    private static final int MAX_PATIENTS = 200;

    private final Patient[] patients = new Patient[MAX_PATIENTS];
    private int count = 0;

    public Patient findByNationalId(String nationalId) {
        if (nationalId == null) {
            return null;
        }
        String id = nationalId.trim();
        for (int i = 0; i < count; i++) {
            Patient p = patients[i];
            if (p != null && id.equals(p.getNationalId())) {
                return p;
            }
        }
        return null;
    }

    public boolean saveOrUpdate(Patient patient) {
        if (patient == null) {
            return false;
        }

        Patient existing = findByNationalId(patient.getNationalId());
        if (existing != null) {
            for (int i = 0; i < count; i++) {
                Patient p = patients[i];
                if (p != null && patient.getNationalId().trim().equals(p.getNationalId())) {
                    patients[i] = patient;
                    return true;
                }
            }
        }

        if (count >= MAX_PATIENTS) {
            throw new IllegalStateException("patients storage is full");
        }

        patients[count++] = patient;
```

HospitalMangementSystem (run) #3 running... × (3 more...) 53:1 INS Windows (CRLF)

- Figure 45: (AppointmentDAO array storage)

HospitalMangementSystem.java [-/A] × PatientDAO.java [-/A] × AppointmentDAO.java [-/A] ×

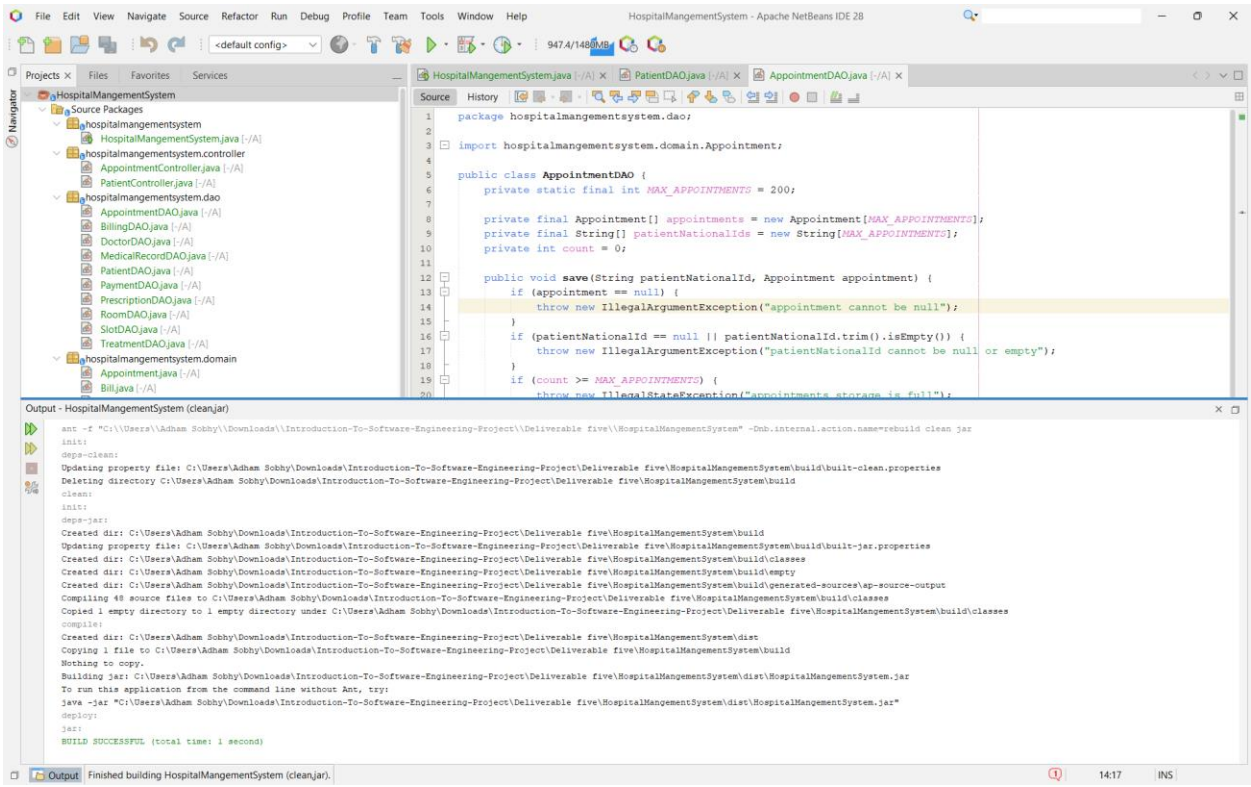
Source History

```java
package hospitalmangementsystem.dao;

import hospitalmangementsystem.domain.Appointment;

public class AppointmentDAO {
    private static final int MAX_APPOINTMENTS = 200;

    private final Appointment[] appointments = new Appointment[MAX_APPOINTMENTS];
    private final String[] patientNationalIds = new String[MAX_APPOINTMENTS];
    private int count = 0;

    public void save(String patientNationalId, Appointment appointment) {
        if (appointment == null) {
            throw new IllegalArgumentException("appointment cannot be null");
        }
        if (patientNationalId == null || patientNationalId.trim().isEmpty()) {
            throw new IllegalArgumentException("patientNationalId cannot be null or empty");
        }
        if (count >= MAX_APPOINTMENTS) {
            throw new IllegalStateException("appointments storage is full");
        }
        appointments[count] = appointment;
        patientNationalIds[count] = patientNationalId.trim();
        count++;
    }

    public Appointment findById(String appointmentId) {
        if (appointmentId == null) {
            return null;
        }
        for (int i = 0; i < count; i++) {
            Appointment a = appointments[i];
            if (a != null && appointmentId.equals(a.getAppointmentId())) {
                return a;
            }
        }
        return null;
    }

    public Appointment[] findUpcomingByPatientNationalId(String patientNationalId) {
        if (patientNationalId == null) {
            return new Appointment[0];
        }

```

HospitalMangementSystem (run) #3 running... × (3 more...) 1:1 INS Windows (CRLF)

10. Evidence Checklist (Screenshots)

- Figure 46: NetBeans Clean and Build → BUILD SUCCESSFUL



11. Jira / GitHub Evidence

12. Conclusion

Deliverable 5 produced a complete use-case driven OO design for HMS. We classified use cases, selected 12 core use cases across complexity levels, and applied CRC/communication/sequence realization techniques accordingly. We consolidated the extracted operations into the final DCD and validated architecture using the package diagram.

Finally, we performed StarUML model-to-code transformation and implemented UC-02 and UC-06 as a runnable NetBeans walking skeleton using strict layered architecture and array-based DAO persistence, satisfying deliverable constraints.

Appendix A — Run Instructions (NetBeans)

Open project HospitalMangementSystem in NetBeans
Right-click project → Clean and Build
Run main: hospitalmangementsystem.HospitalMangementSystem

Menu:
1 → UC-02
2 → UC-06
0 → Exit

Appendix B — Submission Checklist

( ) PDF report with diagrams inserted inline (Figures 1–49)
( ) StarUML .mdj file included
( ) NetBeans project folder included
( ) Build success screenshot included
( ) UC-02 and UC-06 run screenshots included
( ) Jira/GitHub evidence included